LEARNX

A Project Report Submitted to

G.VENKATASWAMY NAIDU COLLEGE (AUTONOMOUS),

KOVILPATTI

In requirement of the partial fulfilment for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS

Submitted by

MS. M. SANKARESWARI
Reg. no: 232CA133

Under the Supervision of

Mrs. M. GANDHIMATHI, M.C.A., M.Phil.,
Assistant professor & Head of Computer Applications

Submitted to

DEPARTMENT OF COMPUTER APPLICATIONS

G. VENKATASWAMY NAIDU COLLEGE (AUTONOMOUS),

(Re-Accredited by NAAC with 'A' Grade)

AFFILIATED TO MANONMANIAM SUNDARANAR UNIVERSITY, TIRUNELVELI

KOVILPATTI - 628 502.

November-2025

CERTIFICATE

This is to certify that the project entitled **"LEARNX"** submitted by **Ms.M.SANKARESWARI** in partial fulfillment for the award of the Degree of **BACHELOR OF COMPUTER APPLICATIONS** to the G.Venkataswamy Naidu College (Autonomous), Kovilpatti affiliated to Manonmaniam Sundaranar University, Tirunelveli is a record of the bonafide work carried out under my guidance and supervision. The results presented in this project report have not been submitted to any other University or Institute for the award of any other degree.

PROJECT GUIDE **HEAD OF THE DEPARTMENT**

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION

I declare that the project entitled **"LEARNX"** is a bonafide work duly completed by me with the guidance and supervision of the **Mrs. M. GANDHIMATHI, MCA., M.Phil., Head & Assistant Professor**. It does not contain any part of the project or thesis submitted by any other candidate to this or any other Institute or University.

All such materials that have been obtained from other sources have been duly acknowledged.

(Signature)

Date **By**

M. SANKARESWARI

(232CA133)

ACKNOWLEDGEMENT

I would like to thank God for all his blessings to finish our project successfully. Next I wish to thank our parents for their financial and moral support and to their gracious blessings.

I wish to place on records our ardent and earnest gratitude to the respected Principal **Dr. M. SUBBULAKSHMI, M.Sc., M.Phil., Ph.D.,** and beloved Director **Dr. B. MAKESH KUMAR, M.SC., M.Phil., Ph.D.,** for providing motivation and encouragement in making the project a truly successful one.

I feel honored and elated to have worked under the guidance of **Mrs. M. GANDHIMATHI, MCA, M.Phil.,** H e a d & Assistant Professor of the department who has been a source of inspiration right from the movement of initiation of this project. I feel heartily grateful for her guidance.

My sincere thanks to our Lab Technician **Mrs. R. BHUVANESHWARI, M.Sc., B.Ed.,** toward his needful help and also we thank our friends for their keen attention and thought suggestion over this project.

Finally, I would like to express our sincere gratitude and special thanks to all our respectable staffs and classmates for encouraging us to complete this project.

S. SANKARESWARI

(232CA133)

CONTENTS

S.NO	INDEX	PAGE NO
1	**INTRODUCTION**	1
2	**PROBLEM DEFINITION & DESCRIPTION** 2.1 PROBLEM DEFINITION 2.2 PROBLEM DESCRIBTION	2
3	**WORKING ENVIRONMENT** 3.1 HARDWARE SPECIFICATION 3.2 SOFTWARE SPECIFICATION	3
4	**SYSTEM ANALYSIS** 4.1 EXISTING SYSTEM 4.2 PROPOSED SYSTEM	5
5	**SYSTEM DESIGN** 5.1 DATA FLOW DIAGRAM	6
6	**SYSTEM IMPLEMENTATION** 6.1 MODULES	7
7	**TESTING** 7.1 UNIT TESTING 7.2 INTEGRATION TESTING 7.3 SECURITY TESTION	10
8	**SOURCE CODE**	12
9	**SCREEN LAYOUT**	59
10	**CONCLUSION**	71
11	**FUTURE ENHANCEMENT**	72
12	**REFERENCE**	73

ABSTRACT

LearnX is an innovative e-commerce-based online learning platform designed to bridge the gap between teachers and students through a digital environment that promotes interactive education. The main objective of LearnX is to create a space where teachers can easily post and manage their courses, while students can conveniently register, learn, and track their academic progress. Each course includes detailed information such as syllabus, duration, and learning materials, ensuring that students gain a complete understanding of what they will learn before enrolling. Students can also submit assignments directly through the platform, making the entire learning and evaluation process streamlined and paperless.

One of the platform's most notable features is its **performance-based certification system**. When a student scores more than 80% in their course assessments, they automatically receive a **course completion certificate**, which serves as proof of their achievement and can be useful for future academic or career opportunities. Teachers, on the other hand, can view and manage which students have enrolled in their courses and monitor their progress efficiently through an intuitive dashboard.

LearnX integrates a **chatbot powered by the Gemini API**, which acts as a 24/7 virtual assistant to answer students' queries, guide them through the platform, and provide immediate help when needed. This ensures that learners receive constant support without delays.

Technically, LearnX is built using **React** for the frontend, providing a fast, responsive, and dynamic user interface. The backend is supported by **Firebase Firestore**, ensuring secure data storage, real-time updates, and seamless user interaction. Overall, LearnX combines education and technology to make online learning more accessible, engaging, and result-oriented. It empowers teachers to share their expertise and helps students learn effectively while earning certificates that validate their hard work and skills.

1. INTRODUCTION

In the digital era, education has rapidly shifted from traditional classrooms to online and hybrid learning models. However, many existing e-learning platforms face challenges such as limited interactivity, lack of flexibility, poor scalability, and weak communication channels between educators and students. Recognizing these challenges, LearnX has been designed as an innovative solution to bridge these gaps and enhance the online learning experience for both learners and educators.

LearnX is a real-time, Firebase-powered web platform that offers a dynamic and interactive environment for education. It enables educators — including teachers, professors, and trainers to create, manage, and customize courses across multiple domains such as Mathematics, Science, Coding, and English. By integrating rich multimedia content such as text, images, and embedded YouTube videos, the platform ensures that lessons are engaging, accessible, and tailored to different learning styles.

For students, LearnX provides an intuitive interface to browse, filter, and preview available courses based on their interests, schedules, and budgets. Registration is made seamless and secure through real-time synchronization and Firebase authentication. Notifications are instantly delivered to educators when students register, ensuring smooth communication and tracking of course enrollments.

Additionally, the platform supports role-based user profiles, allowing for personalized dashboards for students, teachers, and professors. Its scalable architecture ensures the platform can grow along with its user base, making it suitable for schools, colleges, coaching centers, and even corporate training environments.

With the integration of real-time database management, responsive design, and modern cloud infrastructure, LearnX stands out as a next-generation learning solution that simplifies online education while maintaining quality, security, and interactivity.

2. PROBLEM DEFINITION AND DESCRIPTION

2.1 PROBLEM DEFINITION

Traditional online learning platforms often lack interactivity, personalized support, and an easy way to validate student achievements. Many systems do not provide seamless communication between teachers and students or automated certification based on performance. Teachers face difficulties in tracking student participation, while students struggle with instant doubt clarification and progress monitoring. There is a need for a secure, flexible, and user-friendly platform that not only connects teachers and students but also ensures interactive learning, real-time support, assignment evaluation, and performance-based certification.

2.2 PROBLEM DESCRIPTION

In the current digital era, online education has become widespread, but existing platforms often lack features that ensure an engaging and outcome-oriented learning experience. Students frequently face challenges such as limited interaction with educators, lack of instant doubt clarification, and insufficient tracking of their learning progress. Similarly, teachers struggle to effectively manage courses, monitor student performance, and issue valid certifications. Without a structured system, learning outcomes become less impactful, and the process feels incomplete. Therefore, there is a need for an online learning solution that integrates course management, assignment evaluation, performance tracking, and automated certification, along with real-time chatbot support to make the learning process more effective and interactive.

3. WORKING ENVIRONMENT

3.1 HARDWARE SPECIFICATION

The successful development and deployment of the **Learnx** application require appropriate hardware components for both development and usage.

For Development:

1. **Processor**: Intel Core i5 or higher (or AMD Ryzen 5 or higher)
2. **RAM:** Minimum 8GB (Recommended: 16GB for smooth performance)
3. **Storage:** Minimum 256GB SSD (Recommended: 512GB SSD or higher for faster)
4. **Graphics Card:** Integrated GPU (Recommended: Dedicated GPU for faster video)
5. **Operating System:** Windows 10/11, macOS, or Linux
6. **Internet Connection:** Stable highspeed internet for Firebase integration, real-time.

3.2 SOFTWARE SPECIFICATION

The LearnX platform is developed using a modern technology stack combining programming tools, frameworks, and cloud-based services to ensure high performance, scalability, security, and a seamless user experience.

For Development

Programming Languages:

- JavaScript (ES6+) – for building dynamic, interactive components.
- React.js – for responsive and interactive front-end development.

Integrated Development Environment (IDE):

- Visual Studio Code (latest stable version) – for efficient coding and debugging.

Database & Backend Services:

- Firebase Firestore – for a cloud-based, real-time database to store course details, profiles.
- Firebase Authentication – for secure user login and role-based access management.
- Firebase Storage – for storing course images, profile pictures, and other media assets.

Design Tools:

- Figma – for UI/UX prototyping and wireframing of pages and components.

Version Control:

- Git & GitHub – for code management, version tracking, and team collaboration.

APIs & Libraries:

- Google Firebase SDK – for backend services integration.
- React Router – for efficient navigation and routing across pages.
- Material UI / ShadCN Components – for building a modern, clean, and user-friendly interface.

4 SYSTEM ANALYSIS

4.1 EXISTING SYSTEM

Current online learning platforms mainly allow teachers to upload courses and students to enroll, but they lack advanced features such as automated assignment evaluation, performance-based certification, and instant doubt clarification. Interaction between teachers and students is limited, making it difficult for learners to get timely support. Certificates, if provided, are often issued manually without proper validation of student performance. Teachers also face challenges in tracking which students have registered for their courses and monitoring their progress effectively. These limitations reduce the overall learning experience and make the system less engaging and outcome-oriented.

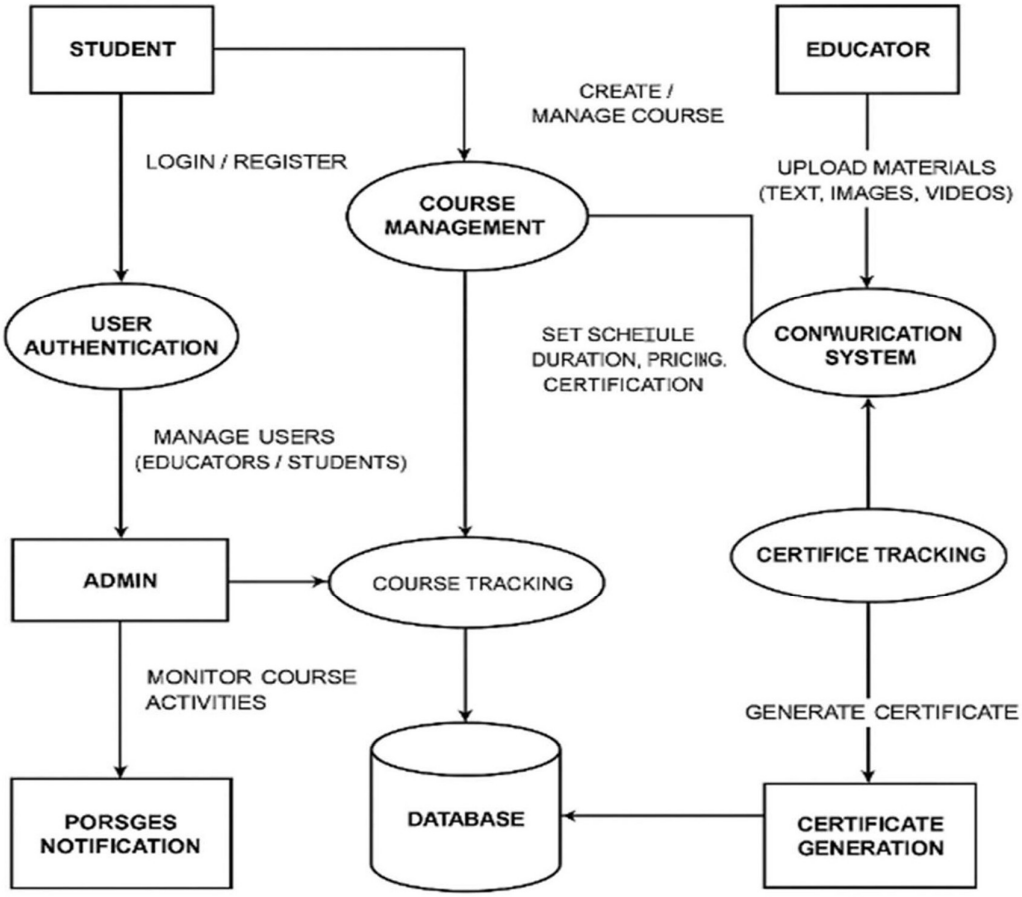
4.2 PROPOSED SYSTEM

The proposed system, LearnX, is an advanced e-learning platform designed to overcome the limitations of existing systems by integrating interactivity, automation, and real-time support. Teachers can create and manage detailed courses with syllabus, assignments, schedules, and certification options, while easily tracking student registrations and monitoring their performance. Students can register, learn, submit assignments, and automatically receive a course completion certificate if they score above 80%. To enhance engagement, the system includes a Gemini API-powered chatbot that provides instant doubt clarification and support. Built using React for the frontend and Firebase Firestore for the backend, LearnX ensures scalability, real-time data management, secure authentication, and an overall simplified yet effective online learning experience.

5. SYSTEM DESIGN

4.3 DATA FLOW DIAGRAM

Data Flow Diagram (DFD) and explanation for Learn-X with the Course Post features integrated.



6 SYSTEM IMPLEMENTATION

LearnX is a web-based learning platform that connects students and teachers through interactive digital education. It allows teachers to create courses with videos, notes, and resources while students can easily access, preview, and register for subjects. With secure login, progress tracking, and digital certificates, LearnX makes learning simple, engaging, and accessible anytime, anywhere.

4.4 MODULES

4.4.1 User Authentication & Database Setup

- Firebase Authentication ensures secure login using Email-Password or Google Sign- In.
- Firestore Database stores collections such as Students, Educators, Courses, Registrations, Certificates, and Notifications.
- Firebase Storage is used for course thumbnails, educator profile pictures, and learning materials (PDFs, images, videos).
- Firestore Security Rules validate user roles and ensure only authorized users (Admin, Educator, Student) can access or modify data.

4.4.2 UI/UX Design & Navigation

- Figma or similar prototyping tools are used for UI wireframes before development.
- A simple web-based dashboard (responsive for mobile/tablet) is provided.

Navigation includes:

- Home: Recent courses, notifications, and announcements.
- Courses: Browse, filter, preview, and register for courses.
- My Learning: Track enrolled courses and progress.
- Resources: Access shared notes, PDFs, and external links.
- Certificates: View and download earned certificates.

4.4.3 Profile Management & Data Handling

- Student Profile includes: Name, Email, Role, Department, and Registered Courses.
- Educator Profile includes: Name, Email, Expertise, and Created Courses.
- Features: Upload profile picture, edit details, and logout.
- Firestore Real-time Listeners ensure immediate UI updates when profile or course details.

4.4.4 Course, Events & Announcements Management

- Course Management
- Educators create/manage courses with details (title, description, syllabus, price, schedule)
- Students can browse courses and register instantly.
- Event Announcements
- Displays details such as event name, date, time, and description.
- Admins can post, edit, or delete events.
- Notification System:
- Sends automated updates about course enrollment, new lessons, or deadlines to students and educators.

4.4.5 Certificate & Payment Integration

- Certificate Management
- Digital certificates are auto-generated for students after completing a course.
- Certificates are stored in Firebase and available for download anytime.
- Payment Integration (Optional for Paid Courses)
- Students view course fees and pay securely through external payment gateway.
- Payment verification stored in Firestore (transaction ID, payment history).

4.4.6 Security & Access Control

Role-Based Authentication:

- Students: Register, access resources, track progress, download certificates.
- Educators: Create/manage courses, upload resources, view student enrollments.
- Admins: Full access (manage users, approve courses, view payments).
- Firestore Security Rules enforce restricted access based on role claims.
- UID-based Authentication ensures that users can only access their own data.
- All sensitive data is encrypted and securely stored in Firebase.

4.4.7 Testing & Debugging

- Tested on multiple browsers (Chrome, Edge, Firefox) and mobile devices for responsiveness.
- Firebase Crashlytics monitors application crashes and backend issues.
- Firebase Analytics tracks user activity for future improvements.
- Testing covers authentication flows, real-time updates, course registration, and notifications.

4.4.8 Deployment & Maintenance

Deployment:

- Hosted on Firebase Hosting (web app).
- Scalable for thousands of users with real-time database support.

Maintenance:

- Continuous security updates for user data protection.
- Feature enhancements based on feedback (e.g., new filters, advanced certificate options).
- Regular performance monitoring and bug fixing.

7 TESTING

Testing ensures that the LearnX application functions correctly, is secure, and provides a seamless learning experience for students, educators, and admins. The testing process helps in identifying bugs, performance issues, and security flaws before deployment, ensuring reliability and scalability of the platform.

1. UNIT TESTING

Unit Testing focuses on testing individual components of LearnX, such as course creation, authentication, database operations, and notifications.

Course Management (Educators & Students)

- Verify that a course is successfully created and stored in Firebase.
- Ensure course details (title, description, schedule, price, certification) are saved correctly.
- Check if students can register but cannot modify educator-created courses.

Resource Sharing (Educators & Students)

- Ensure PDFs, notes, and links uploaded by educators are correctly stored in Firebase Storage.
- Verify that students can download/view but not delete academic resources.
- Confirm real-time updates appear instantly in student dashboards.

Certificate System

- Verify certificates are auto-generated upon course completion.
- Ensure certificates contain correct student details and course info.
- Confirm download links work properly and files are stored securely.

User Authentication (Login & Role-based Access)

- Verify that admins, educators, and students can log in successfully.
- Ensure role-based dashboards are correctly displayed.
- Check that unauthorized access (e.g., student accessing admin panel) is blocked.

2. INTEGRATION TESTING

Integration Testing ensures smooth interaction between modules such as authentication, course management, payments, and notifications.

Examples of Integration Testing:

- Course Enrollment System → Verify that when a student registers, the course is added to their profile and educator notifications are triggered.
- Resource Sharing → Check if educator-uploaded materials are instantly reflected in student dashboards.
- Payment System (if enabled) → Ensure successful transactions update course access in the database; failed payments are handled gracefully.
- User Authentication → Confirm students, educators, and admins see only their allowed navigation options.

SECURITY TESTING

- Firebase Security Rules Testing (High Priority):
- Students should only access their own profile and enrolled courses.
- Educators should only manage their own courses/resources.
- Admins should have full access to all modules.
- Test Firestore Rules using the Simulator to prevent accidental data exposure.
- Avoid insecure rules like allow read, write: if true;. Authentication & Authorization Testing:
- Verify strict role-based permissions (Student / Educator / Admin).
- Ensure students cannot modify admin or educator content.
- Test that session tokens and UIDs are validated correctly.
- Check secure logout and token expiration handling.

8. SOURCE CODE

LANDING PAGE

```
import React from 'react';
import { Link } from 'react-router-dom';
import back from './back.jpg'; // 2ன ்ந       L       ் import 6ச ்த d

function LandingPage() { return (

   <div
     className="flex flex-col items-center justify-center min-h-screen bg-cover bg-center text-white"
     style={{ backgroundImage: url(${back}) }}
   >

     <div className="absolute inset-0 bg-black opacity-60"></div>


     <div className="relative z-10 flex flex-col items-center text-center p-4">
       <h1 className="text-5xl md:text-7xl font-extrabold mb-4"> Welcome to <span
         className="text-blue-400">LearnX</span>
       </h1>
       <p className="text-xl md:text-2xl mb-8 max-w-2xl font-light">
         Your path to a brighter future starts here. Join thousands of learners and begin your journey
         today.
       </p>
       <div className="flex space-x-4">
         <Link to="/signup"
           className="bg-blue-600 hover:bg-blue-700 text-white font-bold py-3 px-8 rounded-full
shadow-lg transition-transform transform hover:scale-105"
         >
           Register
         </Link>
         <Link to="/login"
           className="bg-gray-800 hover:bg-gray-900 text-white font-bold py-3 px-8 rounded-full
shadow-lg transition-transform transform hover:scale-105"
         >
           Login
         </Link>
       </div>
     </div>
   </div>
 );
}

export default LandingPage;
```

REGISTER PAGE

```
import React, { useState } from 'react';
import { createUserWithEmailAndPassword, signInWithPopup, GoogleAuthProvider } from
'firebase/auth'; import { auth } from '../firebase';
import { useNavigate, Link } from 'react-router-dom'; import logo from './logo.jpg';
import signupImage from './image.jpg';

const slideInLeft = {
  animation: 'slide-in-left 0.7s cubic-bezier(0.25, 0.46, 0.45, 0.94) both'
};

function Signup() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [confirmPassword, setConfirmPassword] = useState(''); const [error, setError] = useState('');
  const [loading, setLoading] = useState(false); const navigate = useNavigate();

  const inputClasses = "w-full px-4 py-3 border border-gray-300 rounded-lg focus:outline-none
focus:ring-2 focus:ring-blue-500 transition-all duration-300";
  const buttonClasses = "w-full bg-blue-600 text-white py-3 px-4 rounded-lg font-semibold
hover:bg-blue- 700 transition-colors duration-300 disabled:bg-blue-400";

  async function handleSubmit(e) { e.preventDefault();
    if (password !== confirmPassword) { return setError('Passwords do not match');
    }
    try { setError('');
      setLoading(true);
      await createUserWithEmailAndPassword(auth, email, password); alert("Registered successfully
      with Email & Password!"); navigate('/login');
    } catch (err) {
      setError('Failed to create an account: ' + err.message);
    }
    setLoading(false);
  }

  async function handleGoogleSignup() { try {
      const provider = new GoogleAuthProvider(); await signInWithPopup(auth, provider);
      alert("Registered successfully with Google!"); navigate('/login');
    } catch (err) {
      setError("Google Sign Up failed: " + err.message);
    }
  }


return (
    <div className="flex min-h-screen bg-white">
      {/* Left column - background */}
      <div className="hidden md:flex flex-col items-center justify-center w-1/2 bg-blue-600
relative overflow-hidden" style={slideInLeft}>
        <img src={signupImage} alt="Signup background" className="w-full h-full object-cover
absolute inset-0" />
```

```
      </div>

      {/* Right column - signup form */}
      <div className="flex items-center justify-center w-full md:w-1/2 p-8">
        <div className="w-full max-w-md">
          <div className="flex justify-center mb-6">
            <img src={logo} alt="LearnX Logo" className="rounded-full h-24 w-24 object-cover" />
          </div>
          <h2 className="text-3xl font-bold text-center text-gray-800 mb-2">Create LearnX
          Account</h2>
          <p className="text-center text-gray-500 mb-6">Sign up to get started</p>

          {error && (
            <div className="bg-red-100 text-red-700 p-3 rounded-lg mb-4 text-sm text-center"
            role="alert">
              {error}
            </div>
          )}

          <form onSubmit={handleSubmit} className="space-y-4">
            <div>
              <label className="block text-gray-700 text-sm font-medium mb-1">Email Address</label>
              <input type="email" value={email} onChange={(e) => setEmail(e.target.value)}
                      required className={inputClasses} />
            </div>
            <div>
              <label className="block text-gray-700 text-sm font-medium mb-1">Password</label>
              <input type="password" value={password} onChange={(e) =>
setPassword(e.target.value)} required className={inputClasses} />
            </div>
            <div>
              <label className="block text-gray-700 text-sm font-medium mb-1">Confirm
              Password</label>
              <input type="password" value={confirmPassword} onChange={(e) => setCo
            </div>
            <button type="submit" disabled={loading} className={buttonClasses}>
              {loading ? 'Signing up...' : 'Sign Up'}
            </button>
          </form>

          {/* Google signup button */}
          {/* Google signup button */}
<button onClick={handleGoogleSignup}
  className="mt-4 w-full flex items-center justify-center border border-gray-300 bg-white text-gray-
700 py- 3 px-4 rounded-lg font-medium shadow-sm hover:bg-gray-100 transition-colors duration-
300"
>
  <img

    src="https://www.gstatic.com/firebasejs/ui/2.0.0/images/auth/google.svg"


    alt="Google Logo" className="h-5 w-5 mr-3"
```

```
  />
  Continue with Google
</button>


        <p className="mt-6 text-center text-gray-600"> Already have an account?
          <Link to="/login" className="text-blue-600 font-semibold hover:underline ml-1">Log
          In</Link>
        </p>
      </div>
    </div>

    <style>
      {`
        @keyframes slide-in-left {
          0% { transform: translateX(-1000px); opacity: 0; } 100% { transform: translateX(0); opacity:
          1; }
        }
      `}
    </style>
  </div>
  );
}

export default Signup;
```

LOGIN PAGE

```
import React, { useState } from 'react'; import { signInWithEmailAndPassword, signInWithPopup,
GoogleAuthProvider, sendPasswordResetEmail
} from 'firebase/auth';
import { auth, db } from '../firebase';
import { getDoc, doc } from 'firebase/firestore'; import { useNavigate, Link } from 'react-router-
dom'; import logo from './logo.jpg';
import signupImage from './image.jpg';

const slideInRight = {
  animation: 'slide-in-right 0.7s cubic-bezier(0.25, 0.46, 0.45, 0.94) both'
};

function Login() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState(''); const [error, setError] = useState('');
  const [message, setMessage] = useState('');


  const [loading, setLoading] = useState(false);


  const navigate = useNavigate(); const inputClasses =
    'w-full px-4 py-3 border border-gray-300 rounded-lg focus:outline-none focus:ring-2 focus:ring-
blue-500 transition-all duration-300';
```

```
const buttonClasses =
  'w-full bg-blue-600 text-white py-3 px-4 rounded-lg font-semibold hover:bg-blue-700 transition-
colors duration-300 disabled:bg-blue-400';

async function handleSubmit(e) { e.preventDefault();
  try { setError('');
    setMessage(''); setLoading(true);

    const userCredential = await signInWithEmailAndPassword( auth,
      email, password
    );
    const user = userCredential.user;

    const userDocRef = doc(db, 'profiles', user.uid); const docSnap = await getDoc(userDocRef);

    if (docSnap.exists()) { navigate('/home');
    } else {
      navigate('/create-profile');
    }
  } catch (err) {
    setError('Failed to log in: ' + err.message);
  }
  setLoading(false);
}

async function handleGoogleLogin() { try {
    const provider = new GoogleAuthProvider();
    const result = await signInWithPopup(auth, provider); const user = result.user;

    const userDocRef = doc(db, 'profiles', user.uid); const docSnap = await getDoc(userDocRef);

    if (docSnap.exists()) { navigate('/home');
    } else {
      navigate('/create-profile');
    }
  } catch (err) {
    setError('Google Login failed: ' + err.message);


  }

}

async function handleForgotPassword() { if (!email) {
    setError('Please enter your email first.'); return;
  }
  try { setError('');
    setMessage('');
    await sendPasswordResetEmail(auth, email); setMessage('Password reset link sent to your
    email.');
  } catch (err) {
    setError('Failed to send reset email: ' + err.message);
```

```
    }
  }

  return (
    <div className="flex min-h-screen bg-white">
      {/* Left column - login form */}
      <div className="flex items-center justify-center w-full md:w-1/2 p-8">
        <div className="w-full max-w-md">
          <div className="flex justify-center mb-6">
            <img src={logo}
              alt="DocNur Logo"
              className="rounded-full h-24 w-24 object-cover"
            />
          </div>
          <h2 className="text-3xl font-bold text-center text-gray-800 mb-2"> Welcome Back
          </h2>
          <p className="text-center text-gray-500 mb-6"> Log in to your account
          </p>

          {error && (
            <div
              className="bg-red-100 text-red-700 p-3 rounded-lg mb-4 text-sm text-center" role="alert"
            >
              {error}
            </div>
          )}

          {message && (
            <div
              className="bg-green-100 text-green-700 p-3 rounded-lg mb-4 text-sm text-center"
              role="alert"
            >
              {message}
            </div>
          )}

<form onSubmit={handleSubmit} className="space-y-4">
  <div>
    <label className="block text-gray-700 text-sm font-medium mb-1"> Email Address
    </label>
    <input type="email" value={email}
      onChange={(e) => setEmail(e.target.value)} required
      className={inputClasses}
    />
  </div>
  <div>
    <label className="block text-gray-700 text-sm font-medium mb-1"> Password
    </label>
    <input type="password" value={password}
      onChange={(e) => setPassword(e.target.value)} required
      className={inputClasses}
    />
```

```
  </div>

  {/* Full-width login button */}
  <button type="submit" disabled={loading} className={buttonClasses}>
    {loading ? 'Logging in...' : 'Log In'}
  </button>

  {/* Centered Forgot Password link */}
  <div className="flex justify-center">
    <button type="button"
      onClick={handleForgotPassword}
      className="text-sm text-blue-600 hover:underline mt-2"
    >
      Forgot Password?
    </button>
  </div>
</form>


          {/* Google login button */}
          <button onClick={handleGoogleLogin}
            className="mt-4 w-full flex items-center justify-center border border-gray-300 bg-white
text-gray- 700 py-3 px-4 rounded-lg font-medium shadow-sm hover:bg-gray-100 transition-colors
duration-300"
          >
            <img src="https://www.gstatic.com/firebasejs/ui/2.0.0/images/auth/google.svg" alt="Google
              Logo"


              className="h-5 w-5 mr-3"


            />
            Continue with Google
          </button>

          <p className="mt-6 text-center text-gray-600"> Don't have an account?
            <Link to="/"
              className="text-blue-600 font-semibold hover:underline ml-1"
            >
              Sign Up
            </Link>
          </p>
        </div>
      </div>

      {/* Right column - background */}
      <div
        className="hidden md:flex flex-col items-center justify-center w-1/2 bg-blue-600 relative
overflow- hidden"
        style={slideInRight}
      >
        <img  src={signupImage} alt="Login background"
```

```
          className="w-full h-full object-cover absolute inset-0"
        />
      </div>

      <style>
        {`
          @keyframes slide-in-right {
            0% { transform: translateX(1000px); opacity: 0; } 100% { transform: translateX(0); opacity:
            1; }
          }
        `}
      </style>
    </div>
  );
}

export default Login;
```

POST COURSE PAGE

```
import React, { useState, useEffect } from 'react'; import { useNavigate } from 'react-router-dom';
import { auth, db, storage } from '../firebase';
import { collection, addDoc, doc, getDoc } from 'firebase/firestore'; import { ref, uploadBytes,
getDownloadURL } from 'firebase/storage';

const PostCourse = () => {
  const initialQuizData = () => ({ question: '',
    options: ['', '', '', ''], correctAnswer: ''
  });

  const [step, setStep] = useState(1);
  const [courseData, setCourseData] = useState({ title: '',
    description: '', youtubeVideoLink: '', professorName: '', institution: '', professorBio: '', startDate: '',
    endDate: '',
    category: '', // ✅ New field for category courseImageUrl: '',
    weeks: [{ weekNumber: 1, title: '',
      videoLink: '', assignments: [{
        quiz: Array.from({ length: 5 }, initialQuizData)
      }]
    }]
  });

  const [courseImage, setCourseImage] = useState(null); const [loading, setLoading] =
  useState(false);
  const [error, setError] = useState(''); const navigate = useNavigate();

  useEffect(() => {
    const fetchProfessorInfo = async () => { const user = auth.currentUser;
      if (user) { try {
          const userDocRef = doc(db, 'profiles', user.uid); const docSnap = await getDoc(userDocRef);
          if (docSnap.exists() && (docSnap.data().profession === 'School Teacher' ||
          docSnap.data().profession
```

```
=== 'College Professor')) { setCourseData(prevData => ({
            ...prevData,


            professorName: docSnap.data().name,

            institution: docSnap.data().institution || docSnap.data().city
          }));
        } else {
          alert('Only teachers and professors can post courses.'); navigate('/home');
        }
      } catch (err) {
        console.error("Failed to fetch profile:", err); setError("Failed to load professor details.");
        alert("Failed to load professor details."); navigate('/home');
      }
    } else { navigate('/login');
    }
  };
  fetchProfessorInfo();
}, [navigate]);

const handleNext = () => {
  if (step === 1 && (!courseData.title || !courseData.description ||
  !courseData.youtubeVideoLink ||
!courseData.category || !courseImage)) {
    setError('Please fill in all details, choose a category, and upload a course image for this step.');
    return;
  }
  if (step === 2 && (!courseData.professorBio || !courseData.startDate || !courseData.endDate)) {
     setError('Please fill in all details for this step.');
     return;
  }
  if (step === 3) {
    for (const week of courseData.weeks) { if (!week.title || !week.videoLink) {
       setError('Please fill in all details for each week (title and video link).'); return;
     }
     for (const quiz of week.assignments[0].quiz) {
       if (!quiz.question || quiz.options.some(opt => !opt) || !quiz.correctAnswer) {
         setError('Please complete all 5 MCQs with questions, options, and a correct answer for each
         week.'); return;
       }
     }
    }
  }
  setError(''); setStep(step + 1);
};

const handleBack = () => { setError('');
  setStep(step - 1);
};

const handleWeekChange = (index, field, value) => {
```

```
newWeeks[index][field] = value;
      setCourseData({ ...courseData, weeks: newWeeks });
    };

    const handleMcqChange = (weekIndex, mcqIndex, field, value) => { const newWeeks =
      [...courseData.weeks];
      const quizToUpdate = { ...newWeeks[weekIndex].assignments[0].quiz[mcqIndex] };

      if (field.startsWith('option')) {
        const optionIndex = parseInt(field.replace('option', ''), 10); const newOptions =
        [...quizToUpdate.options]; newOptions[optionIndex] = value; quizToUpdate.options =
        newOptions;
      } else {
        quizToUpdate[field] = value;
      }

      newWeeks[weekIndex].assignments[0].quiz[mcqIndex] = quizToUpdate; setCourseData({
      ...courseData, weeks: newWeeks });
    };

    const addWeek = () => { setCourseData({
        ...courseData,
        weeks: [...courseData.weeks, { weekNumber: courseData.weeks.length + 1, title: '',
          videoLink: '', assignments: [{
            quiz: Array.from({ length: 5 }, initialQuizData)
          }]
        }]
      });
    };

    const removeLastWeek = () => {
      if (courseData.weeks.length > 1) { setCourseData({
          ...courseData,
          weeks: courseData.weeks.slice(0, -1),
        });
      }
    };

    const handleSubmit = async (e) => { e.preventDefault(); setLoading(true);
      setError('');

      // Step 1: Validation
      if (!courseData.title || !courseData.description || !courseData.youtubeVideoLink ||
      !courseData.category ||
  !courseImage) {
          setError('Please fill in all details, choose a category, and upload an image for the course.');


          setLoading(false);


          return;
```

```
    }
    for (const week of courseData.weeks) { if (!week.title || !week.videoLink) {
          setError('Please fill in all details for each week (title and video link).'); setLoading(false);
          return;
        }
        for (const quiz of week.assignments[0].quiz) {
          if (!quiz.question || quiz.options.some(opt => !opt) || !quiz.correctAnswer) {
              setError('Please complete all 5 MCQs with questions, options, and a correct answer for each
              week.'); setLoading(false);
              return;
          }
        }
    }

    try {
      // Step 2: Upload image to Firebase Storage
      const imageRef = ref(storage, course_images/${courseImage.name + new Date().getTime()});
      await uploadBytes(imageRef, courseImage);
      const courseImageUrl = await getDownloadURL(imageRef);

      // Step 3: Add course data to Firestore with image URL const docRef = await
      addDoc(collection(db, 'courses'), {
        ...courseData,
        courseImageUrl: courseImageUrl, authorId: auth.currentUser.uid, createdAt: new Date(),
      });
      console.log("Course posted with ID:", docRef.id); navigate('/courses');
    } catch (err) {
      console.error("Failed to post course:", err); setError('Failed to post course. Please try again.');
    } finally { setLoading(false);
    }
  };

  const renderStep = () => { switch (step) {
      case 1:
        return (
          <div className="space-y-4">
            <h3 className="text-xl font-semibold text-gray-800">Step 1: Course Details</h3>
            <p className="text-sm text-gray-500">Enter the course title, description, and an
introductory video link.</p>

            <input type="text"
              placeholder="Course Title" value={courseData.title}

              onChange={(e) => setCourseData({ ...courseData, title: e.target.value })}


              className="w-full p-3 border rounded-lg" required
            />

            <textarea
              placeholder="Course Description" value={courseData.description}
              onChange={(e) => setCourseData({ ...courseData, description: e.target.value })}
              className="w-full p-3 border rounded-lg"
```

```
          rows="4" required
        ></textarea>

        <input type="text"
          placeholder="YouTube Video Link (Intro)" value={courseData.youtubeVideoLink}
          onChange={(e) => setCourseData({ ...courseData, youtubeVideoLink: e.target.value })}
          className="w-full p-3 border rounded-lg"
          required
        />

        {/* ✅ Category Dropdown */}
        <label className="block text-sm font-medium text-gray-700 mt-4">Select Category:</label>
        <select value={courseData.category}
          onChange={(e) => setCourseData({ ...courseData, category: e.target.value })}
          className="w-full p-3 border rounded-lg"
          required
        >
          <option value="">-- Choose Category --</option>
          <option value="Technical">Technical</option>
          <option value="Non-Technical">Non-Technical</option>
          <option value="Arts">Arts</option>
          <option value="Others">Others</option>
        </select>

        <label className="block text-sm font-medium text-gray-700 mt-4">Course Image:</label>
        <input type="file"
          onChange={(e) => setCourseImage(e.target.files[0])} className="w-full p-3 border
          rounded-lg"
          required
        />
      </div>
    );
  case 2:
    return (
      <div className="space-y-4">
        <h3 className="text-xl font-semibold text-gray-800">Step 2: Professor and Timeline</h3>
        <p className="text-sm text-gray-500">Enter professor details and the course timeline.</p>
        <input type="text" placeholder="Professor Name"
                value={courseData.professorName} className="w-full p-3 border rounded-lg"
disabled />


        <input type="text" placeholder="Institution/Workplace"
        value={courseData.institution}



className="w-full p-3 border rounded-lg" disabled />
        <textarea placeholder="Professor Bio" value={courseData.professorBio} onChange={(e)
=> setCourseData({ ...courseData, professorBio: e.target.value })} className="w-full p-3 border
rounded-lg" rows="4" required ></textarea>
        <label className="block text-sm font-medium text-gray-700 mt-4">Start Date:</label>
        <input type="date" value={courseData.startDate} onChange={(e) => setCourseData({
```

```
...courseData, startDate: e.target.value })} className="w-full p-3 border rounded-lg" required />
          <label className="block text-sm font-medium text-gray-700">End Date:</label>
          <input type="date" value={courseData.endDate} onChange={(e) => setCourseData({
...courseData, endDate: e.target.value })} className="w-full p-3 border rounded-lg" required />
        </div>
      );
    case 3:
      return (
        <div className="space-y-6">
          <h3 className="text-xl font-semibold text-gray-800">Step 3: Weekly Course Content
& Assignments</h3>
          <p className="text-sm text-gray-500">Enter the weekly topics, video links, and a 5-
question MCQ assignment.</p>
          {courseData.weeks.map((week, index) => (
            <div key={index} className="p-4 border rounded-lg bg-gray-50">
              <h4 className="font-semibold text-gray-700 mb-2">Week {week.weekNumber}</h4>
              <input type="text" placeholder="Week Title" value={week.title}
                onChange={(e) => handleWeekChange(index, 'title',
e.target.value)} className="w-full p-2 border rounded-lg mb-2" required
/>
              <input type="text" placeholder="YouTube Video Link" value={week.videoLink}
              onChange={(e)
=> handleWeekChange(index, 'videoLink', e.target.value)} className="w-full p-2 border rounded-lg
mb-2" required />

              <h5 className="font-bold text-gray-800 my-4">MCQ Assignment (5 Questions)</h5>
              {week.assignments[0].quiz.map((mcq, mcqIndex) => (
                <div key={mcqIndex} className="p-3 border border-dashed rounded-lg bg-white mb-
                4">
                  <p className="text-sm font-medium text-gray-700 mb-2">Question {mcqIndex +
                  1}</p>
                  <input type="text" placeholder="Question" value={mcq.question} onChange={(e) =>
handleMcqChange(index, mcqIndex, 'question', e.target.value)} className="w-full p-2 border
rounded-lg mb-2" required />

                  {mcq.options.map((option, optionIndex) => (
                    <input key={optionIndex} type="text" placeholder={Option ${optionIndex + 1}}
value={option} onChange={(e) => handleMcqChange(index, mcqIndex, option${optionIndex},
e.target.value)} className="w-full p-2 border rounded-lg mb-2" required />
                  ))}

                  <input type="text" placeholder="Correct Answer (e.g., 'Option 2')"
value={mcq.correctAnswer} onChange={(e) => handleMcqChange(index, mcqIndex,
'correctAnswer', e.target.value)} className="w- full p-2 border rounded-lg" required />
                </div>
              ))}
            </div>
          ))}
          <div className="flex space-x-2">
            <button type="button" onClick={addWeek} className="flex-1 bg-blue-500 text-white
            font-
```

```
semibold py-2 px-4 rounded-lg hover:bg-blue-600 transition-colors"> Add Week


            </button>
            {courseData.weeks.length > 1 && (
              <button type="button" onClick={removeLastWeek} className="flex-1 bg-red-500 text-
white font-semibold py-2 px-4 rounded-lg hover:bg-red-600 transition-colors">
                Remove Last Week
              </button>
            )}
          </div>
        </div>
      );
    default: return null;
  }
};

 return (
   <div className="flex items-center justify-center min-h-screen bg-gray-100 p-4">
     <div className="bg-white p-8 rounded-2xl shadow-2xl w-full max-w-2xl">
       <h2 className="text-3xl font-bold text-center text-gray-800 mb-6">Post a New Course</h2>
       {error && (
         <div className="bg-red-100 text-red-700 p-3 rounded-lg mb-4 text-sm text-center"
         role="alert">
           {error}
         </div>
       )}
       <form onSubmit={handleSubmit}>
         {renderStep()}
         <div className="flex justify-between mt-6">
           {step > 1 && (
             <button  type="button"  onClick={handleBack}  className="bg-gray-300  text-gray-800
font- semibold py-2 px-6 rounded-lg hover:bg-gray-400 transition-colors">
               Back
             </button>
           )}
           {step < 3 ? (
             <button type="button" onClick={handleNext} className="ml-auto bg-blue-600 text-
white font- semibold py-2 px-6 rounded-lg hover:bg-blue-700 transition-colors">
               Next
             </button>
           ) : (
             <button type="submit" disabled={loading} className={ml-auto ${loading ? 'bg-green-
300' : 'bg- green-600'} text-white font-semibold py-2 px-6 rounded-lg hover:bg-green-700
transition-colors}>
               {loading ? 'Posting...' : 'Post Course'}
             </button>
           )}
         </div>
       </form>
     </div>
   </div>
```

```
  );
};

export default PostCourse;
```

COURSE LIST

```
import React, { useState, useEffect } from 'react'; import { db } from '../firebase';
import { collection, onSnapshot, query } from 'firebase/firestore'; import { Link } from 'react-router-
dom';

const CourseList = () => {
  const [courses, setCourses] = useState([]); const [loading, setLoading] = useState(true); const
  [error, setError] = useState('');
  const [selectedCategory, setSelectedCategory] = useState('All');

  useEffect(() => {
    const q = query(collection(db, 'courses'));

    const unsubscribe = onSnapshot( q,
      (snapshot) => {
        const coursesArray = snapshot.docs.map((doc) => ({ id: doc.id,
          ...doc.data(),
        }));
        setCourses(coursesArray); setLoading(false);
      },
      (err) => {
        console.error('Failed to fetch courses:', err); setError('Failed to load courses.');
        setLoading(false);
      }
    );

    return () => unsubscribe();
  }, []);

  const filteredCourses = selectedCategory === 'All'
      ? courses
      : courses.filter((course) => course.category === selectedCategory);

  if (loading) { return (
      <div className="flex items-center justify-center min-h-screen bg-gray-100">
        <p className="text-xl text-gray-600">Loading courses...</p>
      </div>
    );
  }
```

```
if (error) { return (
    <div className="flex items-center justify-center min-h-screen bg-gray-100">
      <p className="text-red-500">{error}</p>


    </div>



  );
}

return (
  <div className="min-h-screen bg-gray-100 p-4 md:p-8">
    <div className="container mx-auto">
      <h2 className="text-3xl font-bold text-center text-gray-800 mb-8"> All Available Courses
      </h2>

      {/* 🔥 Category Filter */}
      <div className="flex justify-center mb-6 space-x-4">
        {['All', 'Technical', 'Non-Technical', 'Arts', 'Others'].map((cat) => (
          <button key={cat}
            onClick={() => setSelectedCategory(cat)}
            className={`px-4 py-2 rounded-full text-sm font-medium transition ${ selectedCategory
              === cat
                ? 'bg-blue-600 text-white'
                : 'bg-gray-200 text-gray-700 hover:bg-gray-300'
            }`}
          >
            {cat}
          </button>
        ))}
      </div>

      {filteredCourses.length === 0 ? (
        <p className="text-center text-gray-600">
          No courses found for "{selectedCategory}" category.
        </p>
      ) : (
        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
          {filteredCourses.map((course) => (
            <div key={course.id}
              className="bg-white rounded-xl shadow-lg hover:shadow-xl transition-shadow duration-
              300 flex
```

```
flex-col">
{/* Course Image */}
{course.courseImageUrl && (
  <img src={course.courseImageUrl} alt={course.title}
    className="w-full h-48 object-cover rounded-t-xl"
  />
)}

<div className="p-6 flex flex-col justify-between flex-grow">
  <div>
    <h3 className="text-xl font-bold text-gray-800 mb-2">
      {course.title}
    </h3>
    <p className="text-sm text-gray-500 mb-2">


                Professor: <span className="font-semibold">{course.professorName}</span> |{' '}
                {course.institution}
              </p>

              {/* 🔥 Category Badge */}
              {course.category && (
                <span className="inline-block px-3 py-1 text-xs font-semibold bg-blue-100 text-
blue-700 rounded-full mb-4">
                  {course.category}
                </span>
              )}

              <p className="text-gray-600 line-clamp-3">{course.description}</p>
            </div>
            <div className="mt-4 text-center">
              <Link to={/courses/${course.id}}
                className="text-blue-600 hover:text-blue-800 font-semibold inline-block"
              >
                View More Details
              </Link>
            </div>
          </div>
        </div>
      ))}
    </div>
  )}
 </div>
 </div>
);
};

export default CourseList;
```

COURSEDETAILS

```
import React, { useState, useEffect } from 'react'; import { useParams, Link } from 'react-router-
dom';
import { db, auth } from '../firebase';
import { doc, getDoc } from 'firebase/firestore'; import { onAuthStateChanged } from
'firebase/auth';

const CourseDetails = () => { const { courseId } = useParams();
  const [course, setCourse] = useState(null); const [loading, setLoading] = useState(true); const
  [error, setError] = useState('');
  const [user, setUser] = useState(null);
  const [isRegistered, setIsRegistered] = useState(false);

  useEffect(() => {
    const unsubscribe = onAuthStateChanged(auth, async (currentUser) => { setUser(currentUser);

      if (currentUser) {
        const courseRef = doc(db, 'courses', courseId); const docSnap = await getDoc(courseRef);
        if (
          docSnap.exists() && docSnap.data().registeredUsers?.includes(currentUser.uid)
        ) {
          setIsRegistered(true);
        }
      }
    });
    return () => unsubscribe();
  }, [courseId]);

  useEffect(() => {
    const fetchCourse = async () => { try {
        const docRef = doc(db, 'courses', courseId); const docSnap = await getDoc(docRef);

        if (docSnap.exists()) { setCourse(docSnap.data());
        } else {
          setError('Course not found.');
        }
      } catch (err) {
        console.error('Failed to fetch course details:', err); setError('Failed to load course details.');
      } finally { setLoading(false);
      }
    };
    fetchCourse();
  }, [courseId]);

  if (loading) { return (
      <div className="flex items-center justify-center min-h-screen bg-gray-100">
        <p className="text-xl text-gray-600">Loading course details...</p>
      </div>


    );
  }
```

```
if (error) { return (
    <div className="flex items-center justify-center min-h-screen bg-gray-100">
      <p className="text-red-500">{error}</p>
    </div>
  );
}

const getYouTubeEmbedUrl = (url) => { if (!url) return null;
  const regExp =


    /^.(youtu.be\/|v\/|u\/\w\/|embed\/|watch\?v=|\&v=)([^#\&\?]).*/; const match = url.match(regExp);
  return match && match[2].length === 11
    ? https://www.youtube.com/embed/${match[2]}
    : null;
};

return (
  <div className="min-h-screen bg-gray-100 p-4 md:p-8">
    <div className="container mx-auto bg-white p-6 rounded-xl shadow-lg">
      <div className="text-center mb-8">
        {/* 🔥 Category Badge */}
        {course.category && (
          <span className="inline-block px-4 py-1 text-sm font-semibold bg-blue-100 text-blue-700
rounded- full mb-4">
            {course.category}
          </span>
        )}

        <h2 className="text-4xl font-extrabold text-gray-800 mb-2">
          {course.title}
        </h2>
        <p className="text-md text-gray-600">
          Professor: <span className="font-semibold">{course.professorName}</span> |{' '}
          {course.institution}
        </p>
      </div>

      <div className="grid grid-cols-1 md:grid-cols-3 gap-8">
        <div className="md:col-span-2 space-y-6">
          <div className="aspect-w-16 aspect-h-9 bg-gray-200 rounded-lg overflow-hidden">
            {getYouTubeEmbedUrl(course.youtubeVideoLink) ? (
              <iframe src={getYouTubeEmbedUrl(course.youtubeVideoLink)} title="YouTube video
                player"
                frameBorder="0"
                allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-
                picture" allowFullScreen
              ></iframe>
            ) : (


              <div className="flex items-center justify-center h-full text-gray-500"> Video not
```

```
        available
      </div>
    )}
  </div>

  <div>
    <h3 className="text-2xl font-bold text-gray-800 mb-2"> About the Course
    </h3>
    <p className="text-gray-600 leading-relaxed">{course.description}</p>
  </div>

  {!isRegistered && (

    <div>
      <h3 className="text-2xl font-bold text-gray-800 mb-2"> Weekly Course Details
      </h3>
      <p className="text-gray-600">
        Register for the course to view the weekly content.
      </p>
    </div>
  )}
</div>

<div className="md:col-span-1 space-y-6">
  <div className="p-4 bg-gray-50 rounded-lg shadow-sm">
    <h3 className="text-xl font-bold text-gray-800 mb-2"> Course Summary
    </h3>
    <ul className="text-gray-600 space-y-1">
      <li>
        <span className="font-semibold">Status:</span>{' '}
        {course.startDate ? 'Upcoming' : 'Ongoing'}
      </li>
      <li>
        <span className="font-semibold">Duration:</span>{' '}
        {course.weeks?.length}  weeks
      </li>
      <li>
        <span className="font-semibold">Start Date:</span>{' '}
        {course.startDate}
      </li>
      <li>
        <span className="font-semibold">End Date:</span>{' '}
        {course.endDate}
      </li>
    </ul>
  </div>
  <div className="p-4 bg-gray-50 rounded-lg shadow-sm">
    <h3 className="text-xl font-bold text-gray-800 mb-2"> About the Professor
    </h3>
    <p className="text-gray-600 leading-relaxed">
      {course.professorBio}
```

```
            </p>
          </div>
          {isRegistered ? (
            <Link
              to={/my-courses/${courseId}/week/1}
              className="w-full text-center text-white font-semibold py-3 px-6 rounded-lg bg-
green-600 hover:bg-green-700 transition-colors block"
            >
              Go to Course
            </Link>
          ) : (
            <Link
              to={/register-course/${courseId}}
              disabled={!user}
              className={`w-full text-center text-white font-semibold py-3 px-6 rounded-lg
              transition-colors

    !user
    ? 'bg-blue-400 cursor-not-allowed'
    : 'bg-blue-600 hover:bg-blue-700'
  }`}
>
  Join Now
</Link>


          )}
        </div>
      </div>
    </div>
  </div>
  );
};

export default CourseDetails;
```

COURSEWEEK

```
import React, { useState, useEffect } from 'react';
import { Link, useParams, useNavigate } from 'react-router-dom';
import { db, auth } from '../firebase'; // 2 p ச ƒ6ச ṁnṅniL«. path dwjn dÔக ṅni©ȧa«. import {
doc, getDoc, updateDoc, setDoc } from "firebase/firestore";
import { onAuthStateChanged } from 'firebase/auth';

// Courseweek.js 8க wṅ2d இ ṅ8nw« RatingModal aȧp ஒ ц ц$n component 8ச jÔக ṅni©ȧa«.
// இ « nnன j  wL  $iL      ¯pÚத «D d$ṅA©6ச ṁn rத pBp«.

const RatingModal = ({ onSubmit, onCancel }) => { const [rating, setRating] = useState(0);
  const [review, setReview] = useState('');

  const renderStars = () => { const stars = [];
    for (let i = 1; i <= 5; i++) { stars.push(
```

```
        <svg key={i}
          className={`h-8 w-8 cursor-pointer transition-colors duration-200 ${

          i <= rating ? 'text-yellow-400' : 'text-gray-300'
          }`}
          onClick={() => setRating(i)} fill="currentColor" viewBox="0 0 20 20"
        >
          <path d="M9.049 2.927c.3-.921 1.603-.921 1.902 0l1.286 3.961a1 1 0 00.95.691h4.161c.969
          0 1.371
1.24.588 1.81l-3.374 2.454a1 1 0 00-.364 1.118l1.286 3.961c.3.921-.755 1.688-1.54 1.118l-3.374-
2.454a1 1
0 00-1.176 0l-3.374 2.454c-.784.57-1.84-.197-1.54-1.118l1.286-3.961a1 1 0 00-.364-1.118L2.094
9.389c-
.784-.57-.382-1.81.588-1.81h4.161a1 1 0 00.95-.691l1.286-3.961z" />
        </svg>
      );

    }
    return stars;
  };

  const handleSubmit = (e) => { e.preventDefault();
    if (rating === 0) {
      alert("Please select a star rating."); return;
    }


    onSubmit(rating, review);
  };

  return (
    <div className="fixed inset-0 bg-gray-600 bg-opacity-75 z-50 flex items-center justify-center p-
    4">
      <div className="bg-white rounded-2xl shadow-xl p-8 max-w-lg w-full">
        <h3 className="text-2xl font-bold text-gray-800 mb-4 text-center">Give a Rating &
        Review</h3>
        <p className="text-gray-600 mb-6 text-center">Congratulations on completing the course!
Please share your feedback.</p>
        <form onSubmit={handleSubmit} className="space-y-6">
          <div className="flex justify-center items-center space-x-2 mb-4">
            {renderStars()}
          </div>
          <div>
            <label htmlFor="review" className="block text-gray-700 font-semibold mb-
2">Your Review</label>
            <textarea id="review" value={review}
              onChange={(e) => setReview(e.target.value)}
              className="w-full h-32 p-4 border border-gray-300 rounded-lg focus:outline-none
focus:ring-2 focus:ring-indigo-500"
              placeholder="Write your thoughts about the course..."
            ></textarea>
          </div>
          <div className="flex justify-end space-x-4">
```

```
          <button type="button" onClick={onCancel}
            className="px-6 py-3 rounded-lg font-bold text-gray-700 bg-gray-200 hover:bg-gray-
300 transition"
          >
            Cancel
          </button>
          <button type="submit"
            className="px-6 py-3 rounded-lg font-bold text-white bg-indigo-600 hover:bg-indigo-
700 transition"
          >
            Submit
          </button>
        </div>
      </form>
    </div>
  </div>
);
};


// Puthiya Week Navigation Component
const WeekNavigation = ({ courseId, currentWeek, totalWeeks, userProgress }) => { const weeks =
  Array.from({ length: totalWeeks }, (_, i) => i + 1);

  return (
    <div className="flex flex-col gap-2 p-4 bg-gray-200 rounded-xl shadow-inner min-w-[250px]">
      <h4 className="text-xl font-bold text-gray-800 mb-2">Course Content</h4>
      {weeks.map((week) => {
        const isUnlocked = userProgress?.currentWeek >= week; const isCurrent = currentWeek ===
        week;
        const isCompleted = userProgress?.completedWeeks.includes(week);

        const linkClasses = isUnlocked
          ? (isCurrent
            ? "bg-indigo-600 text-white shadow-lg border-2 border-indigo-700"
            : (isCompleted
              ? "bg-green-500 text-white hover:bg-green-600"
                                                        : "bg-blue-500 text-white hover:bg-blue-600"))
                                                      : "bg-gray-400 text-gray-700 cursor-not-allowed";

        return (
          <Link key={week}
            to={isUnlocked ? /my-courses/${courseId}/week/${week} : '#'}
            className={flex items-center p-4 rounded-lg font-bold transition-all duration-300
transform hover:scale-105 ${linkClasses}}
            onClick={(e) => !isUnlocked && e.preventDefault()}
          >
            Week {week}
          </Link>
        );
      })}
    </div>
```

```
  );
};


const CourseWeek = () => {
  const { courseId, weekNumber } = useParams(); const navigate = useNavigate();
  const currentWeek = parseInt(weekNumber, 10);

  const [course, setCourse] = useState(null); const [loading, setLoading] = useState(true); const
  [error, setError] = useState('');
  const [user, setUser] = useState(null);
  const [userProgress, setUserProgress] = useState(null); const [quizAnswers, setQuizAnswers] =
  useState({});


  const [quizScore, setQuizScore] = useState(null);

  const [quizSubmitted, setQuizSubmitted] = useState(false); const [isVideoWatched,
  setIsVideoWatched] = useState(false); const [message, setMessage] = useState('');
  // ч$N a (state): 8µipṁ    pb        ক wiL
  const [showRatingModal, setShowRatingModal] = useState(false);

  useEffect(() => {
    const fetchCourseAndProgress = async () => {
      const unsubscribe = onAuthStateChanged(auth, async (currentUser) => { if (!currentUser) {
        navigate('/login'); return;
      }
      setUser(currentUser);

      try {
        const courseRef = doc(db, 'courses', courseId); const courseSnap = await getDoc(courseRef);

        if (!courseSnap.exists()) { setError('Course not found.'); setLoading(false);
          return;
        }
        const courseData = courseSnap.data(); setCourse(courseData);

        const userProgressRef = doc(db, 'userProgress', ${currentUser.uid}_${courseId}); const
        progressSnap = await getDoc(userProgressRef);

        if (progressSnap.exists()) {
          const progressData = progressSnap.data(); setUserProgress(progressData);

          if (progressData.completedWeeks.includes(currentWeek)) { setIsVideoWatched(true);
          }

          if (currentWeek > progressData.currentWeek) {
            setError('This week is not yet unlocked. Please complete the previous week.');
          }
        } else {
          setError('You are not registered for this course.');
        }
      } catch (err) {
```

```
        console.error("Failed to fetch data:", err); setError('Failed to load course content.');
      } finally { setLoading(false);
      }
    });
    return () => unsubscribe();
  };
  fetchCourseAndProgress();



}, [courseId, weekNumber, navigate, currentWeek]);

const handleVideoWatched = () => { setIsVideoWatched(true);
};

const handleQuizAnswerChange = (mcqIndex, answer) => { setQuizAnswers(prevAnswers => ({
    ...prevAnswers, [mcqIndex]: answer
  }));
};

const submitQuiz = async (e) => { e.preventDefault();
  let correctCount = 0;
  const week = course.weeks[currentWeek - 1]; if (week?.assignments?.[0]?.quiz) {
    week.assignments[0].quiz.forEach((mcq, index) => { if (quizAnswers[index] ===
      mcq.correctAnswer) { correctCount++;
      }
    });
  }

  setQuizScore(correctCount); setQuizSubmitted(true);

  if (correctCount >= 3) { await updateProgress();
  } else {
    setMessage("You need to score at least 3 out of 5 to pass. Please review the video and try
    again.");
  }
};

const updateProgress = async () => { if (!user || !userProgress) return;

  try {
    const userProgressRef = doc(db, "userProgress", ${user.uid}_${courseId});

    const newCompletedWeeks = [...userProgress.completedWeeks]; if
    (!newCompletedWeeks.includes(currentWeek)) { newCompletedWeeks.push(currentWeek);
    }

    let newCurrentWeek = userProgress.currentWeek; if (currentWeek ===
    userProgress.currentWeek) { newCurrentWeek = userProgress.currentWeek + 1;
    }

    await setDoc( userProgressRef,
```

```
      {



        uid: user.uid, courseId: courseId,
        completedWeeks: newCompletedWeeks, currentWeek: newCurrentWeek,
      },
      { merge: true }
    );

    setUserProgress({
      ...userProgress,
      completedWeeks: newCompletedWeeks, currentWeek: newCurrentWeek,
    });

    // ч$N  п¿  : пwLÚ$iLD ¯p¿ततत w a      Ö ச ƒпwjÔக pD
    // пwLÚ$iLD ¯p¿«6iLwd, 8µipṁ  pb      Ô க wÙ2Ôக pD if (newCompletedWeeks.length
    === course.weeks.length) { setMessage(Congratulations! You have completed the entire
    course!); setShowRatingModal(true);
    } else { setMessage(
        Congratulations! You have completed Week ${currentWeek}. Week ${newCurrentWeek} is
now unlocked!
      );
    }

  } catch (err) {
    console.error("Failed to update user progress:", err); setError("Failed to update your progress.");
  }
};


// ч$N 6ச NdпW©: D$ṅA© Dj²D 6Djசன  Ö ச Djṅ2Ôக const
handleRatingAndReviewSubmit = async (rating, review) => { try {
    const reviewDocRef = doc(db, "courseRatingsAndReviews", ${user.uid}_${courseId}); await
    setDoc(reviewDocRef, {
      userId: user.uid, courseId: courseId, rating,
      review,
      timestamp: new Date()
    });
    setMessage("Thank you for your valuable feedback!");
  } catch (error) {
    console.error("Error submitting rating and review:", error); setMessage("Failed to submit your
    feedback. Please try again.");
  } finally { setShowRatingModal(false);
  }
};


const getYouTubeEmbedUrl = (url) => {
```

```
  if (!url) return null;
  const regExp = /^.(youtu.be\/|v\/|u\/\w\/|embed\/|watch\?v=|\&v=)([^#\&\?]).*/; const match =
  url.match(regExp);
  return (match && match[2].length === 11) ? https://www.youtube.com/embed/${match[2]} : null;
};

if (loading) { return (
    <div className="flex items-center justify-center min-h-screen bg-gray-100">
      <p className="text-xl text-gray-600">Loading course week...</p>
    </div>
  );
}

if (error) { return (
    <div className="flex items-center justify-center min-h-screen bg-gray-100">
      <p className="text-red-500">{error}</p>
    </div>
  );
}

if (!course || !course.weeks || !course.weeks[currentWeek - 1]) { return (
    <div className="flex items-center justify-center min-h-screen bg-gray-100">
      <p className="text-xl text-gray-600">Course content for this week is not available.</p>
    </div>
  );
}

const week = course.weeks[currentWeek - 1];
const videoEmbedUrl = week ? getYouTubeEmbedUrl(week.videoLink) : null; const hasQuiz =
week?.assignments?.[0]?.quiz?.length > 0;
const isWeekCompleted = userProgress?.completedWeeks.includes(currentWeek); const
isQuizPassed = quizSubmitted && quizScore >= 3;

return (
  <div className="min-h-screen bg-gray-100 p-4 md:p-8 font-sans">
    <div className="container mx-auto max-w-full bg-white p-6 rounded-3xl shadow-2xl flex
flex-col md:flex-row gap-8">

      {showRatingModal && (
       <RatingModal onSubmit={handleRatingAndReviewSubmit} onCancel={() =>
        setShowRatingModal(false)}
       />
      )}

      {course?.weeks && (
       <WeekNavigation courseId={courseId} currentWeek={currentWeek}


        totalWeeks={course.weeks.length} userProgress={userProgress}

       />
```

```
      )}

      <div className="flex-1">
        <h2 className="text-3xl font-extrabold text-gray-800 mb-2">{course.title}</h2>
        <h3     className="text-xl      font-semibold     text-gray-700      mb-6">Week
                {week.weekNumber}:
{week.title}</h3>

        {message && (
         <div className="fixed inset-0 bg-gray-600 bg-opacity-50 overflow-y-auto h-full w-full z-
50 flex justify-center items-center">
           <div className={p-8 rounded-xl shadow-lg text-center mx-4 md:mx-auto ${isQuizPassed
? 'bg- green-50 text-green-800' : 'bg-red-50 text-red-800'}}>
             <h4 className="text-2xl font-bold mb-4">{isQuizPassed ? 'Success!' : 'Oops!'}</h4>
             <p className="text-lg mb-6">{message}</p>
             <button
               onClick={() => setMessage('')}
               className={px-6    py-2    rounded-lg    font-bold    text-white    transition-transform
                                  transform hover:scale-105 ${isQuizPassed ? 'bg-green-600
hover:bg-green-700' : 'bg-red-600 hover:bg-red-700'}}
             >
               Close
             </button>
           </div>
         </div>
        )}

        <div className="space-y-8">
          <div className="bg-gray-50 p-6 rounded-2xl shadow-inner">
            <h4 className="text-2xl font-bold text-gray-800 mb-4">Weekly Lecture</h4>
            <div className="relative" style={{ paddingBottom: '56.25%' }}>
              {videoEmbedUrl ? (
                <iframe
                  className="absolute top-0 left-0 w-full h-full rounded-xl" src={videoEmbedUrl}
                  title="Weekly video player" frameBorder="0"
                  low="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope;
                  picture-in-

   allowFullScreen
  ></iframe>
) : (
  <div className="flex items-center justify-center h-full text-gray-500 rounded-xl">Video not


ilable</div>
              )}
            </div>
            {!isVideoWatched && (
              <button onClick={handleVideoWatched}
                className="mt-6 w-full md:w-auto px-6 py-3 bg-indigo-600 text-white font-bold
rounded-lg shadow-lg hover:bg-indigo-700 transition duration-300 transform hover:scale-105"
              >
                Mark as Watched
```

```
            </button>
          )}



        </div>

        {isVideoWatched && hasQuiz && (
          <div className="bg-white p-8 rounded-2xl shadow-xl border border-gray-200">
            <h4 className="text-2xl font-extrabold text-gray-800 mb-6">Assignment: Quiz</h4>

            {isWeekCompleted ? (
              <div className="bg-green-50 text-green-700 p-6 rounded-xl text-center shadow-md">
                <p className="font-bold text-lg">You have already completed this week's
                assignment.</p>
              </div>
            ) : (
              <form onSubmit={submitQuiz} className="space-y-8">
                {week.assignments[0].quiz.map((mcq, index) => (
                  <div key={index} className="mb-6 p-6 border border-gray-200 rounded-xl bg-
gray-50 shadow-md">
                    <p className="font-semibold text-lg text-gray-800 mb-4">
                      <span className="font-bold text-indigo-600">{index + 1}.</span> {mcq.question}
                    </p>
                    <div className="space-y-3">
                      {mcq.options.map((option, optIndex) => (
                        <label key={optIndex} className="flex items-center p-4 rounded-lg cursor-
pointer transition-all duration-200 hover:bg-indigo-100 hover:shadow-sm group">
                          <input type="radio"
                            name={mcq-${index}} value={option}
                            checked={quizAnswers[index] === option}
                            onChange={(e) => handleQuizAnswerChange(index, e.target.value)}
                            className="form-radio h-5 w-5 text-indigo-600 border-gray-300 focus:ring-
indigo-500
group-hover:ring-indigo-500"
                            required
                          />
                          <span className="ml-4 text-gray-700 font-medium group-hover:text-
indigo-800 transition-colors duration-200">{option}</span>
                        </label>
                      ))}
                    </div>
                  </div>
                ))}
                <div className="flex justify-end">
                  <button


                    type="submit"
                    className="px-8 py-4 bg-indigo-600 text-white font-extrabold text-lg rounded-xl
shadow-lg hover:bg-indigo-700 transition duration-300 transform hover:scale-105"
                    disabled={quizSubmitted}
                  >
```

```
                {quizSubmitted ? 'Quiz Submitted!' : 'Submit Quiz'}
              </button>
            </div>
          </form>
        )}
      </div>
    )}


    {quizSubmitted && !isWeekCompleted && (
      <div className={p-6 rounded-2xl shadow-lg border-l-4 ${isQuizPassed ? 'bg-green-100
text-green- 800 border-green-500' : 'bg-red-100 text-red-800 border-red-500'}}>
        <p className="font-bold text-xl mb-2">Quiz Result: {isQuizPassed ? 'Passed!' :
        'Failed.'}</p>
        <p className="text-lg">Your score is <span className="font-
extrabold">{quizScore}</span> out of {week.assignments[0].quiz.length}.</p>
        {!isQuizPassed && <p className="mt-2 text-md">Please review the video and try again to
unlock the next week.</p>}
      </div>
    )}
  </div>
 </div>
</div>
</div>
);
};

export default CourseWeek;
```

HISTORY

```
import React, { useEffect, useState } from "react"; import { auth, db } from "../firebase";
import { onAuthStateChanged } from "firebase/auth";
import { collection, query, where, getDocs, doc, getDoc } from "firebase/firestore"; import
html2canvas from "html2canvas";
import jsPDF from "jspdf";
import logo from "./logo.jpg"; // 🔥 add your logo path here

const History = () => {
  const [user, setUser] = useState(null);
  const [completedCourses, setCompletedCourses] = useState([]); const [loading, setLoading] =
  useState(true);
  const [selectedCourse, setSelectedCourse] = useState(null);

  useEffect(() => {
    const unsubscribe = onAuthStateChanged(auth, async (currentUser) => { if (currentUser) {
      setUser(currentUser);
      await fetchCompletedCourses(currentUser.uid);
    } else { setUser(null);


      setCompletedCourses([]);
    }
```

```
    setLoading(false);
  });

  return () => unsubscribe();
}, []);

const fetchCompletedCourses = async (uid) => { try {
    const q = query(collection(db, "userProgress"), where("uid", "==", uid)); const querySnap =
    await getDocs(q);


    const courseData = await Promise.all( querySnap.docs.map(async (docSnap) => { const
      progress = docSnap.data();
        const courseId = progress.courseId;

        const courseRef = doc(db, "courses", courseId); const courseDoc = await getDoc(courseRef);

        if (courseDoc.exists()) {
          const course = courseDoc.data();
          const totalWeeks = Array.isArray(course.weeks) ? course.weeks.length : 0;

          if (progress.completedWeeks.length === totalWeeks && totalWeeks > 0) { return { id:
            courseId, ...course, progress };
          }
        }
        return null;
      })
    );

    setCompletedCourses(courseData.filter(Boolean));
  } catch (err) {
    console.error("Error fetching history:", err);
  }
};

// ✅ Download as PDF
const downloadCertificate = () => {
  const input = document.getElementById("certificate"); html2canvas(input, { scale: 3
  }).then((canvas) => { const imgData = canvas.toDataURL("image/png"); const pdf = new
  jsPDF("landscape", "pt", "a4"); const width = pdf.internal.pageSize.getWidth();  const height =
  pdf.internal.pageSize.getHeight(); pdf.addImage(imgData, "PNG", 0, 0, width, height);
    pdf.save(${user.displayName || "student"}_${selectedCourse.title}_certificate.pdf);
  });
};

if (loading) { return (
    <div className="flex justify-center items-center min-h-screen">


      <p className="text-xl text-gray-600">Loading your history...</p>
    </div>
  );
}
```

```
if (!user) { return (
    <div className="flex justify-center items-center min-h-screen">
      <p className="text-xl text-red-500">Please login to view your history.</p>
    </div>
  );


}

return (
  <div className="min-h-screen bg-gray-100 p-6">
    <h2 className="text-3xl font-bold text-gray-800 mb-6 text-center">
      ✅ Completed Courses
    </h2>
    {completedCourses.length === 0 ? (
      <p className="text-center text-gray-600">No courses completed yet.</p>
    ) : (
      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
        {completedCourses.map((course) => (
          <div key={course.id} className="bg-white shadow-lg rounded-xl p-6">
            {course.courseImageUrl && (
              <img src={course.courseImageUrl} alt={course.title}
                className="w-full h-40 object-cover rounded-lg mb-4"
              />
            )}
            <h3 className="text-xl font-bold mb-2">{course.title}</h3>
            <p className="text-sm text-gray-500 mb-2">
              Professor: {course.professorName} | {course.institution}
            </p>
            <p className="text-green-600 font-semibold mb-2">🎉 Completed!</p>
            <button
              className="bg-blue-600 text-white py-1 px-3 rounded hover:bg-blue-700" onClick={()
              => setSelectedCourse(course)}
            >
              View Certificate
            </button>
          </div>
        ))}
      </div>
    )}

    {/* ✅ Certificate Modal */}
    {selectedCourse && (
      <div
        className="fixed inset-0 flex items-center justify-center bg-black bg-opacity-60 z-50"
        onClick={() => setSelectedCourse(null)} // 🔥 close when background clicked
      >
        <div


          className="bg-white rounded-xl shadow-xl p-8 w-11/12 max-w-5xl relative" onClick={(e)
          => e.stopPropagation()} // ⊘ prevent closing when clicking inside
        >
```

```
<button
  className="absolute top-3 right-3 text-gray-600 hover:text-red-600" onClick={() =>
  setSelectedCourse(null)}
>
  ✕
</button>

<div


  id="certificate"
  className="border-[12px] border-yellow-500 shadow-2xl p-12 rounded-xl bg-gradient-to-
br from- white to-gray-50 text-center font-serif"
>
  {/* Logo */}
  <img src={logo} alt="logo" className="w-24 mx-auto mb-4" />

  {/* Title */}
  <h1 className="text-5xl font-extrabold text-yellow-600 mb-4 tracking-wide uppercase">
    Certificate of Achievement
  </h1>

  {/* Divider */}
  <div className="w-32 border-b-4 border-yellow-500 mx-auto mb-6"></div>

  <p className="text-lg text-gray-700 mb-3 italic"> This is proudly presented to
  </p>

  {/* Student Name */}
  <h2 className="text-4xl font-extrabold text-gray-900 mb-4 uppercase">
    {user.displayName || user.email}
  </h2>

  <p className="text-lg text-gray-700 mb-3"> For successfully completing the course
  </p>

  {/* Course Name */}
  <h3 className="text-3xl italic text-blue-800 mb-8"> "{selectedCourse.title}"
  </h3>

  {/* Institution */}
  <p className="text-gray-600 mb-8">
    LearnX - Department of BCA, G. Venkataswamy Naidu College
  </p>

  {/* Signatures */}
  <div className="flex justify-between px-20 mt-12">
    <div className="text-left">
      <div className="border-t-2 border-gray-600 w-40 mb-2"></div>
      <p className="text-gray-800 font-semibold">M. Gandhimathi</p>
```

```
              <p className="text-sm text-gray-600">HOD</p>
            </div>
            <div className="text-right">
              <div className="border-t-2 border-gray-600 w-40 mb-2 ml-auto"></div>
              <p className="text-gray-800 font-semibold">Venkatasalapathi</p>
              <p className="text-sm text-gray-600">Director</p>
            </div>
          </div>
        </div>

        <div className="mt-6 flex justify-center">


          <button onClick={downloadCertificate}
            className="bg-green-600 text-white px-6 py-2 rounded-lg hover:bg-green-700"
          >
            Download PDF
          </button>
        </div>
      </div>
    </div>
  )}
 </div>
);
};

export default History;
```

REGISTER COURSE

```
// src/components/RegisterCourse.js

import React, { useState, useEffect } from "react";
import { useParams, useNavigate } from "react-router-dom"; import { db, auth } from "../firebase";
import {  doc, getDoc, updateDoc, arrayUnion, setDoc,
} from "firebase/firestore";
import { onAuthStateChanged } from "firebase/auth";

const RegisterCourse = () => { const { courseId } = useParams(); const navigate = useNavigate();
  const [user, setUser] = useState(null);  const [course, setCourse] = useState(null); const [loading,
  setLoading] = useState(true);
  const [registrationLoading, setRegistrationLoading] = useState(false); const [error, setError] =
  useState("");
  const [userData, setUserData] = useState({ name: "",
    email: "",
```

```
  collegeOrSchool: "", dateOfBirth: "", currentStudy: "", graduationYear: "", motivation: "",
  experience: "",
});

// 🔥 Check auth state useEffect(() => {
  const unsubscribe = onAuthStateChanged(auth, (currentUser) => {

    if (!currentUser) {
      alert("Please log in to register for a course."); navigate("/login");
    } else { setUser(currentUser); setUserData((prev) => ({
        ...prev,
        name: currentUser.displayName || "", email: currentUser.email || "",
      }));
    }
  });
  return () => unsubscribe();
}, [navigate]);

// 🔥 Fetch course details useEffect(() => {
  const fetchCourse = async () => { if (!courseId) return;
    try {
      const docRef = doc(db, "courses", courseId); const docSnap = await getDoc(docRef);
      if (docSnap.exists()) { setCourse(docSnap.data());
      } else {
        setError("Course not found.");
      }
    } catch (err) {
      setError("Failed to load course details.");
    } finally { setLoading(false);
    }
  };
  fetchCourse();
}, [courseId]);

// 🔥 Handle registration
const handleRegister = async (e) => { e.preventDefault();
  if (!user || !course) return;

  setRegistrationLoading(true); setError("");


  try {
    const courseRef = doc(db, "courses", courseId); const userProfileRef = doc(db, "profiles",
    user.uid);
    const userProgressRef = doc(db, "userProgress", ${user.uid}_${courseId});

    // ✅ Add user to course registeredUsers array await updateDoc(courseRef, { registeredUsers:
    arrayUnion(user.uid),
    });
```

```
    // ✅ Add courseId to user's profile await updateDoc(userProfileRef, {
      registeredCourses: arrayUnion(courseId),
    });

    // ✅ Initialize user progress await setDoc( userProgressRef,
      {
        uid: user.uid, courseId: courseId, completedWeeks: [], currentWeek: 1,
      },
      { merge: true }
    );

    // ✅ Save registration details under "registrations" collection const registrationRef = doc(
      db, "registrations",
      ${user.uid}_${courseId}
    );
    await setDoc(registrationRef, { uid: user.uid,
      courseId: courseId, courseName: course.title,
      ...userData,
      registeredAt: new Date(),
    });

    alert("✅ You have successfully registered for the course!"); navigate(/courses/${courseId}); // Go
    back to course details
  } catch (err) {
    console.error("Failed to register for course:", err); setError(Registration failed: ${err.message});
  } finally { setRegistrationLoading(false);
  }
};

// 🔄 Loading state
if (loading || !user || !course) { return (
    <div className="flex items-center justify-center min-h-screen bg-gray-100">


      <p className="text-xl text-gray-600">Loading registration form...</p>
    </div>
  );
}

// 🔄 Error state if (error) { return (
    <div className="flex items-center justify-center min-h-screen bg-gray-100">

      <p className="text-red-500">{error}</p>
    </div>
  );
}

// 🔄 Form UI return (
  <div className="min-h-screen bg-gray-100 p-4 md:p-8 flex items-center justify-center">
    <div className="bg-white p-8 rounded-xl shadow-lg w-full max-w-2xl">
      <h2 className="text-3xl font-bold text-gray-800 mb-4"> Register for {course.title}
      </h2>
```

```
<p className="text-gray-600 mb-6">
  Please fill out the form below to complete your registration.
</p>

<form onSubmit={handleRegister}>
  <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
    {/* Name */}
    <div>
      <label className="block text-gray-700 text-sm font-bold mb-2"> Name
      </label>
      <input type="text"
        className="shadow border rounded w-full py-2 px-3 text-gray-700"
        value={userData.name}
        onChange={(e) =>
          setUserData({ ...userData, name: e.target.value })
        }
        required
      />
    </div>

    {/* Email */}
    <div>
      <label className="block text-gray-700 text-sm font-bold mb-2"> Email
      </label>
      <input type="email"
        className="shadow border rounded w-full py-2 px-3 text-gray-700"
        value={userData.email}
        onChange={(e) =>
          setUserData({ ...userData, email: e.target.value })

        }


        required
      />
    </div>

    {/* College/School */}
    <div>
      <label className="block text-gray-700 text-sm font-bold mb-2"> College or School

      </label>
      <input type="text"
        className="shadow border rounded w-full py-2 px-3 text-gray-700"
        value={userData.collegeOrSchool}
        onChange={(e) =>
          setUserData({ ...userData, collegeOrSchool: e.target.value })
        }
        required
      />
    </div>

    {/* DOB */}
```

```
<div>
  <label className="block text-gray-700 text-sm font-bold mb-2"> Date of Birth
  </label>
  <input type="date"
    className="shadow border rounded w-full py-2 px-3 text-gray-700"
    value={userData.dateOfBirth}
    onChange={(e) =>
      setUserData({ ...userData, dateOfBirth: e.target.value })
    }
    required
  />
</div>

{/* Current Study */}
<div>
  <label className="block text-gray-700 text-sm font-bold mb-2"> What are you currently
    studying?
  </label>
  <input type="text"
    className="shadow border rounded w-full py-2 px-3 text-gray-700"
    value={userData.currentStudy}
    onChange={(e) =>
      setUserData({ ...userData, currentStudy: e.target.value })
    }
    required
  />
</div>

{/* Graduation Year */}
<div>
  <label className="block text-gray-700 text-sm font-bold mb-2">


    When will you graduate?
  </label>
  <input type="number"
    className="shadow border rounded w-full py-2 px-3 text-gray-700"
    value={userData.graduationYear}
    onChange={(e) =>

      setUserData({ ...userData, graduationYear: e.target.value })
    }
    required
  />
</div>

{/* Motivation */}
<div className="col-span-2">
  <label className="block text-gray-700 text-sm font-bold mb-2"> Why do you want to join
    this course?
  </label>
  <textarea
    className="shadow border rounded w-full py-2 px-3 text-gray-700"
```

```
            value={userData.motivation}
            onChange={(e) =>
              setUserData({ ...userData, motivation: e.target.value })
            }
            rows="3" required
          />
        </div>

        {/* Experience */}
        <div className="col-span-2">
          <label className="block text-gray-700 text-sm font-bold mb-2"> What is your prior
            experience in this field?
          </label>
          <textarea
            className="shadow border rounded w-full py-2 px-3 text-gray-700"
            value={userData.experience}
            onChange={(e) =>
              setUserData({ ...userData, experience: e.target.value })
            }
            rows="3" required
          />
        </div>
      </div>

      {/* Buttons */}
      <div className="flex items-center justify-between mt-6">
        <button type="button"
          className="bg-gray-500 hover:bg-gray-700 text-white font-bold py-2 px-4 rounded"
          onClick={() => navigate(-1)}
        >
          Cancel
        </button>


        <button type="submit"
          disabled={registrationLoading}
          className={`bg-blue-600 text-white font-bold py-2 px-4 rounded ${ registrationLoading
              ? "bg-blue-400 cursor-not-allowed"


              : "hover:bg-blue-700"
          }`}
        >
          {registrationLoading ? "Registering..." : "Register"}
        </button>
      </div>
    </form>
  </div>
</div>
);
};
```

```
export default RegisterCourse;
```

MYLEARNING

```
import React, { useEffect, useState } from "react"; import { auth, db } from "../firebase";
import { onAuthStateChanged } from "firebase/auth";
import { collection, query, where, getDocs, doc, getDoc } from "firebase/firestore"; import { Link }
from "react-router-dom";

const MyLearning = () => {
  const [user, setUser] = useState(null);
  const [myCourses, setMyCourses] = useState([]); const [loading, setLoading] = useState(true);

  useEffect(() => {
    const unsubscribe = onAuthStateChanged(auth, async (currentUser) => { if (currentUser) {
        setUser(currentUser);
        await fetchMyCourses(currentUser.uid);
      } else { setUser(null); setMyCourses([]);
      }
      setLoading(false);
    });

    return () => unsubscribe();
  }, []);

  const fetchMyCourses = async (uid) => { try {
      const q = query(collection(db, "userProgress"), where("uid", "==", uid)); const querySnap =
      await getDocs(q);


      const courseData = [];
      for (const docSnap of querySnap.docs) { const progress = docSnap.data(); const courseId =
        progress.courseId;

        const courseRef = doc(db, "courses", courseId); const courseDoc = await getDoc(courseRef);

        if (courseDoc.exists()) { courseData.push({
            id: courseId,
            ...courseDoc.data(), progress,
          });
        }
      }
      setMyCourses(courseData);
    } catch (err) {
      console.error("Error fetching my courses:", err);
    }
  };

  if (loading) { return (
      <div className="flex justify-center items-center min-h-screen">
        <p className="text-xl text-gray-600">Loading your courses...</p>
      </div>
    );
```

```
  }

  if (!user) { return (
    <div className="flex justify-center items-center min-h-screen">
      <p className="text-xl text-red-500">Please login to view your learning.</p>
    </div>
  );
  }

  return (
  <div className="min-h-screen bg-gray-100 p-6">
    <h2 className="text-3xl font-bold text-gray-800 mb-6 text-center">📚 My Learning</h2>
    {myCourses.length === 0 ? (
      <p className="text-center text-gray-600">You have not joined any course yet.</p>
    ) : (
      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
        {myCourses
         .map((course) => {
           const isCompleted = course.progress.completedWeeks.length === course.weeks.length;
           return (
             <div key={course.id} className="bg-white shadow-lg rounded-xl p-6 flex flex-col
justify- between">
               {course.courseImageUrl && (
                <img src={course.courseImageUrl} alt={course.title}


                 className="w-full h-40 object-cover rounded-lg mb-4"
                />
               )}
               <h3 className="text-xl font-bold mb-2">{course.title}</h3>
               <p className="text-sm text-gray-500 mb-4">


                Progress: {course.progress.completedWeeks.length}/{course.weeks.length} weeks
               </p>
               {isCompleted ? (
                <span className="text-center py-2 px-4 bg-green-600 text-white rounded-
lg">Course Completed!</span>
               ) : (
                <Link
                 to={/my-courses/${course.id}/week/${course.progress.currentWeek}}
                 className="text-center py-2 px-4 bg-blue-600 text-white rounded-lg hover:bg-blue-
                 700"
                >
                 Continue Learning
                </Link>
               )}
             </div>
           );
         })}
      </div>
    )}
```

```
    </div>
  );
};

export default MyLearning;
```

PROFESSOR HISTORY

```
// src/components/ProfessorHistory.js

import React, { useEffect, useState } from "react"; import { auth, db } from "../firebase";
import { collection, query, where, getDocs, doc, getDoc } from "firebase/firestore"; import {
useNavigate } from "react-router-dom";

const ProfessorHistory = () => {
  const [courses, setCourses] = useState([]); const [loading, setLoading] = useState(true); const
  [error, setError] = useState("");
  const navigate = useNavigate();

  useEffect(() => {
    const fetchProfessorCourses = async () => { try {
      const user = auth.currentUser; if (!user) {
        navigate("/login"); return;
      }

      const q = query(collection(db, "courses"), where("authorId", "==", user.uid));

      const querySnapshot = await getDocs(q); const coursesData = [];

      for (const courseDoc of querySnapshot.docs) {
        const courseData = { id: courseDoc.id, ...courseDoc.data() };


        const regQ = query(collection(db, "registrations"), where("courseId", "==", courseDoc.id));
        const regSnap = await getDocs(regQ);

        const registrations = [];
        for (const regDoc of regSnap.docs) { const regData = regDoc.data();
          const userProfileRef = doc(db, "profiles", regData.uid); const userProfileSnap = await
          getDoc(userProfileRef);

          registrations.push({
            ...regData,
            userProfile: userProfileSnap.exists() ? userProfileSnap.data() : null,
          });
        }

        coursesData.push({
          ...courseData, registrations,
        });
      }

      setCourses(coursesData);
```

```
    } catch (err) {
      console.error("Failed to fetch professor history:", err); setError("⚠ Failed to load history. Please
      try again.");
    } finally { setLoading(false);
    }
  };

  fetchProfessorCourses();
}, [navigate]);

if (loading) { return (
    <div className="flex items-center justify-center min-h-screen bg-gray-100">
      <p className="text-gray-600 text-xl animate-pulse">Loading history...</p>
    </div>
  );
}

if (error) { return (
    <div className="flex items-center justify-center min-h-screen bg-gray-100">
      <p className="text-red-500 text-lg">{error}</p>
    </div>
  );
}


return (
  <div className="min-h-screen bg-gray-50 p-6 md:p-12">
    <h2 className="text-4xl font-extrabold text-gray-800 mb-10 text-center">
      📜 Course Registration History


    </h2>

    {courses.length === 0 ? (
      <div className="text-center bg-white p-8 rounded-xl shadow-md border">
        <p className="text-gray-600 text-lg">You haven't posted any courses yet.</p>
      </div>
    ) : (
      <div className="space-y-10">
        {courses.map((course) => (
          <div key={course.id}
            className="bg-white rounded-2xl shadow-lg border border-gray-200 hover:shadow-xl
transition duration-300"
          >
            {/* Course Header */}
            <div className="p-6 border-b border-gray-200 relative">
              <div className="absolute left-0 top-0 h-full w-2 bg-blue-600 rounded-l-2xl"></div>
              <h3 className="text-2xl font-bold text-blue-700 mb-2">{course.title}</h3>
              <p className="text-gray-600">{course.description}</p>

              <span className="absolute top-6 right-6 bg-blue-100 text-blue-700 text-sm font-
semibold px-3 py-1 rounded-full">
```

```
            {course.registrations.length}  Students
          </span>
        </div>

        {/* Registered Students */}
        <div className="p-6">
          <h4 className="text-xl font-semibold text-gray-800 mb-4 flex items-center">
            👥 Registered Students
          </h4>

          {course.registrations.length === 0 ? (
            <p className="text-gray-500 italic">No students have registered yet.</p>
          ) : (
            <ul className="grid grid-cols-1 md:grid-cols-2 gap-5">
              {course.registrations.map((reg, idx) => (
                <li
                  key={idx}
                  className="p-5 border rounded-xl bg-gradient-to-r from-gray-50 to-gray-100
shadow-sm hover:shadow-md transition"
                >
                  <p className="text-gray-900">
                    <span className="font-semibold">Name:</span> {reg.name}
                  </p>
                  <p className="text-gray-900">
                    <span className="font-semibold">Email:</span> {reg.email}
                  </p>
                  <p className="text-gray-900">
                    <span className="font-semibold">College/School:</span> {reg.collegeOrSchool}


                  </p>
                  <p className="text-gray-900">
                    <span className="font-semibold">Graduation Year:</span> {reg.graduationYear}
                  </p>

                  <p className="text-gray-900">
                    <span className="font-semibold">Motivation:</span> {reg.motivation}
                  </p>
                  {reg.userProfile && (
                    <p className="text-sm text-gray-500 mt-2 italic"> (Profile:
                      {reg.userProfile.profession || "N/A"})
                    </p>
                  )}
                </li>
              ))}
            </ul>
          )}
        </div>
      </div>
    ))}
  </div>
)}
</div>
```

```
  );
};

export default ProfessorHistory;
```

CHATBOT

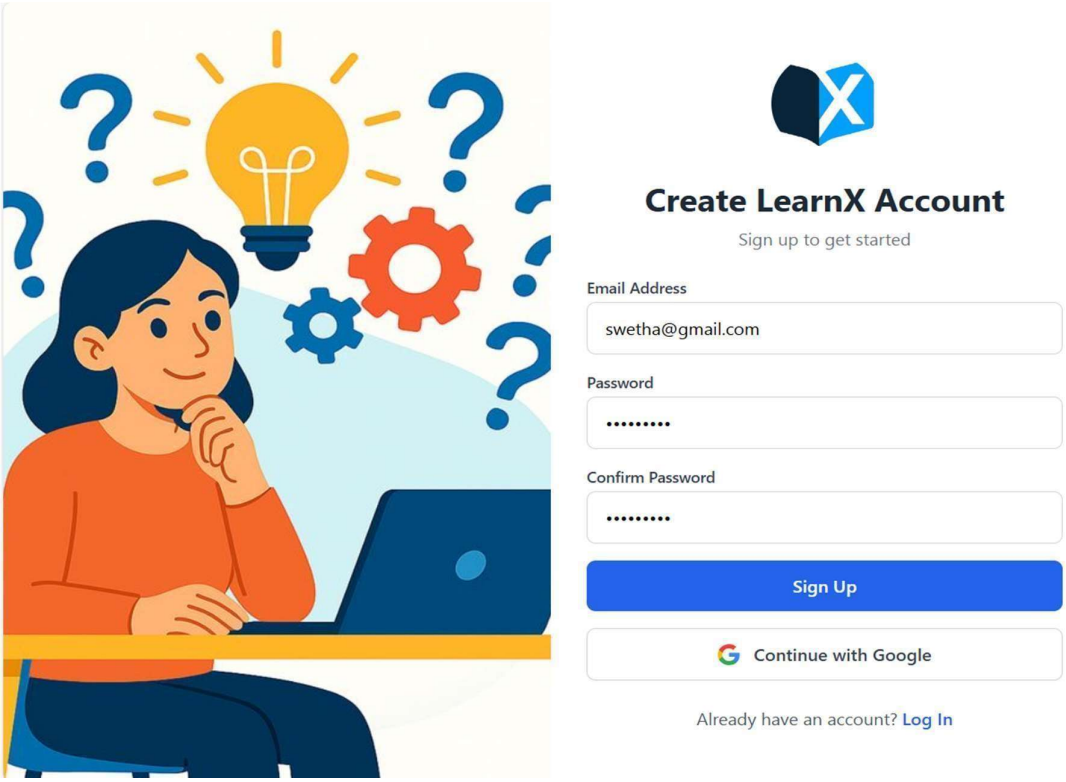
```
// src/components/Chatbot.js
import React, { useState, useRef, useEffect } from "react"; import { GoogleGenerativeAI } from
"@google/generative-ai"; import { MessageCircle, X, Send } from "lucide-react";

const Chatbot = () => {
  const [isOpen, setIsOpen] = useState(false); const [messages, setMessages] = useState([
    { role: "bot", text: "👋 Hi! I'm LearX AI. How can I help you today?" },
  ]);
  const [input, setInput] = useState("");
  const [loading, setLoading] = useState(false); const chatEndRef = useRef(null);

  // 🔥 Gemini API Setup
  const genAI = new GoogleGenerativeAI(process.env.REACT_APP_GEMINI_API_KEY); const
  model = genAI.getGenerativeModel({ model: "gemini-1.5-flash" });

  // ✅ Auto-scroll to latest message useEffect(() => {
    chatEndRef.current?.scrollIntoView({ behavior: "smooth" });
  }, [messages, loading]);

  const sendMessage = async () => { if (!input.trim()) return;


    const newMessages = [...messages, { role: "user", text: input }]; setMessages(newMessages);
    setInput("");


    setLoading(true); try {
      const result = await model.generateContent(input); const response = await result.response.text();
      setMessages([...newMessages, { role: "bot", text: response }]);
    } catch (err) {
      console.error("Gemini API Error:", err); setMessages([
        ...newMessages,
        { role: "bot", text: "⚠️ Sorry, I couldn't fetch a response." },
      ]);
    } finally { setLoading(false);
    }
  };

  return (
    <div>
      {/* Floating Button */}
      {!isOpen && (
        <button
          className="fixed bottom-6 right-6 bg-gradient-to-r from-purple-500 to-indigo-600 p-4
                    rounded-full shadow-xl hover:scale-110 transition-transform text-white"
```

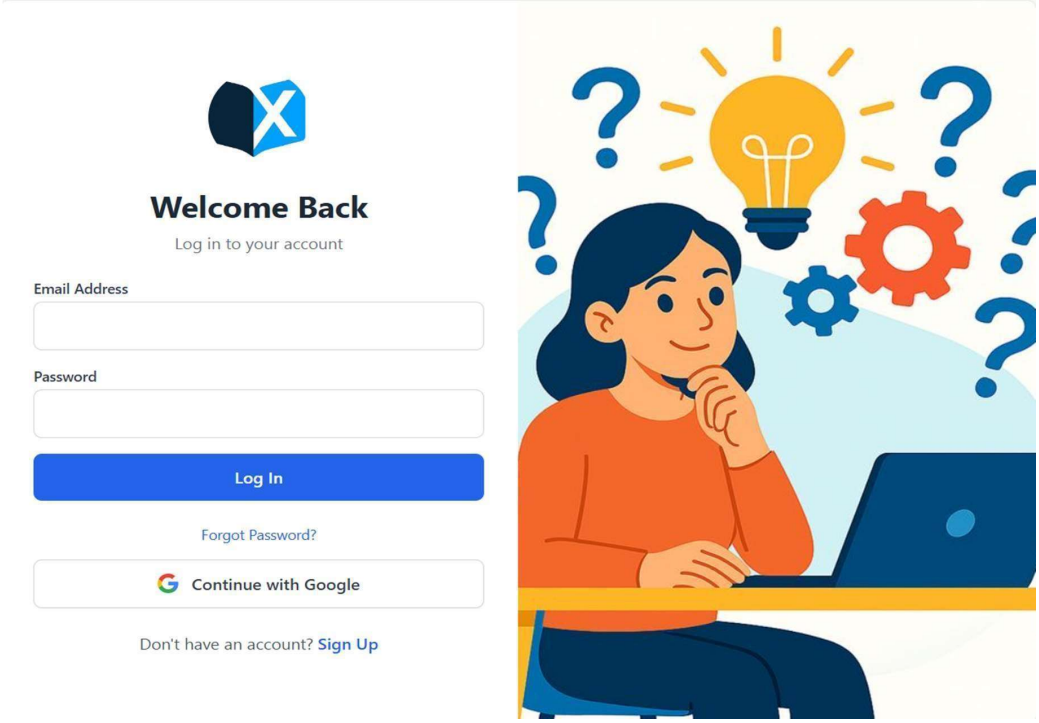
```
    onClick={() => setIsOpen(true)}
  >
    <MessageCircle size={28} />
  </button>
)}

{/* Chat Window */}
{isOpen && (
  <div
    className="fixed bottom-6 right-6 w-96 h-[500px] bg-white dark:bg-gray-900 shadow-2xl
              rounded-2xl flex flex-col overflow-hidden border border-gray-200"
  >
    {/* Header */}
    <div className="bg-gradient-to-r from-purple-500 to-indigo-600 text-white flex justify-
                  between items-center p-4">
      <span className="font-semibold text-lg">💬 LearX AI Assistant</span>
      <button
        className="hover:bg-white/20 rounded-full p-1 transition" onClick={() =>
        setIsOpen(false)}
      >
        <X size={22} />
      </button>
    </div>

    {/* Messages */}
    <div className="flex-1 p-4 overflow-y-auto space-y-3 text-sm bg-gray-50 dark:bg-gray-
    800">
      {messages.map((msg, idx) => (


        <div key={idx}
          className={`max-w-[80%] p-3 rounded-2xl shadow-sm ${


            msg.role === "user"
              ? "ml-auto bg-purple-600 text-white rounded-br-none"
              : "mr-auto bg-gray-200 dark:bg-gray-700 text-gray-900 dark:text-white rounded-bl-
              none"
          }`}
        >
          {msg.text}
        </div>
      ))}
      {loading && (
        <div className="text-gray-400 italic">✍ LearX AI is typing...</div>
      )}
      <div ref={chatEndRef} />
    </div>

    {/* Input Box */}
    <div className="border-t p-3 flex gap-2 bg-white dark:bg-gray-900">
      <input type="text"
        className="flex-1 border rounded-xl px-4 py-2 text-sm focus:outline-none focus:ring-2
```

```
                focus:ring-purple-500 dark:bg-gray-800 dark:text-white"
            placeholder="Type your message..." value={input}
            onChange={(e) => setInput(e.target.value)} onKeyDown={(e) => e.key === "Enter" &&
            sendMessage()}
          />
          <button onClick={sendMessage}
            className="bg-gradient-to-r from-purple-500 to-indigo-600 text-white
                px-4 py-2 rounded-xl hover:opacity-90 transition flex items-center gap-1"
          >
            <Send size={16} /> Send
          </button>
        </div>
      </div>
    )}
  </div>
);
};

export default Chatbot;
```

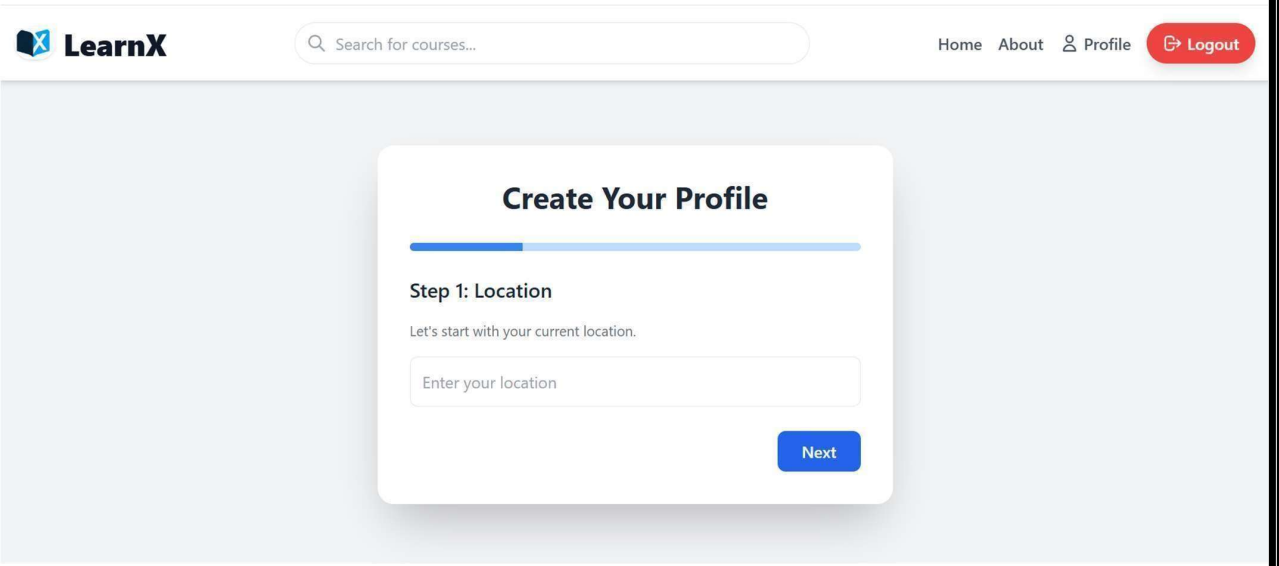
9. SCREEN LAYOUT

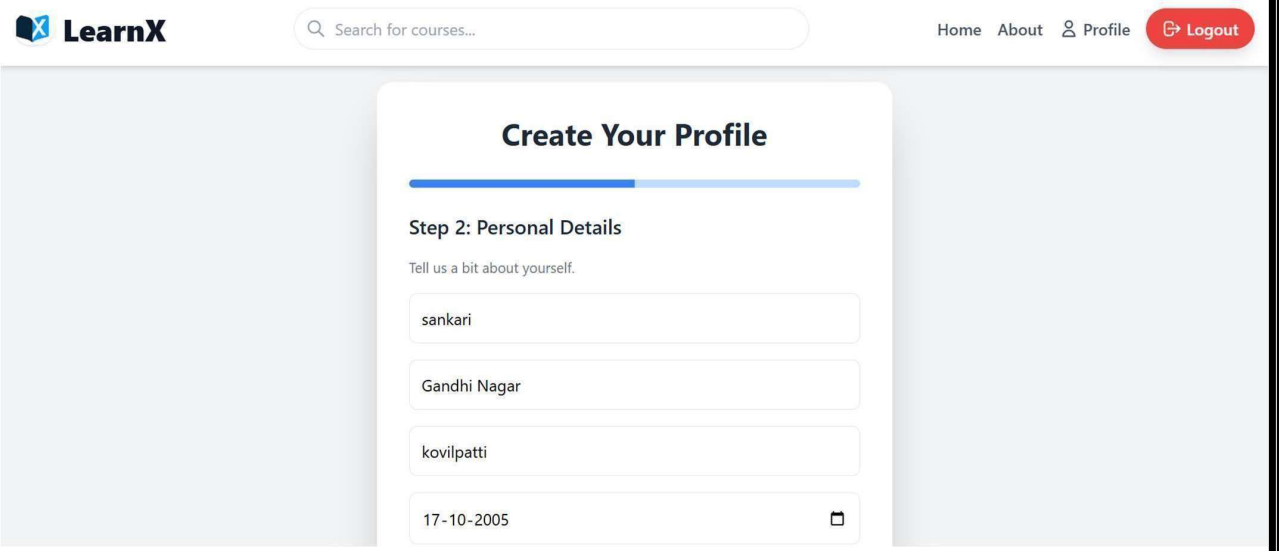
REGISTER AND LOGIN PAGE

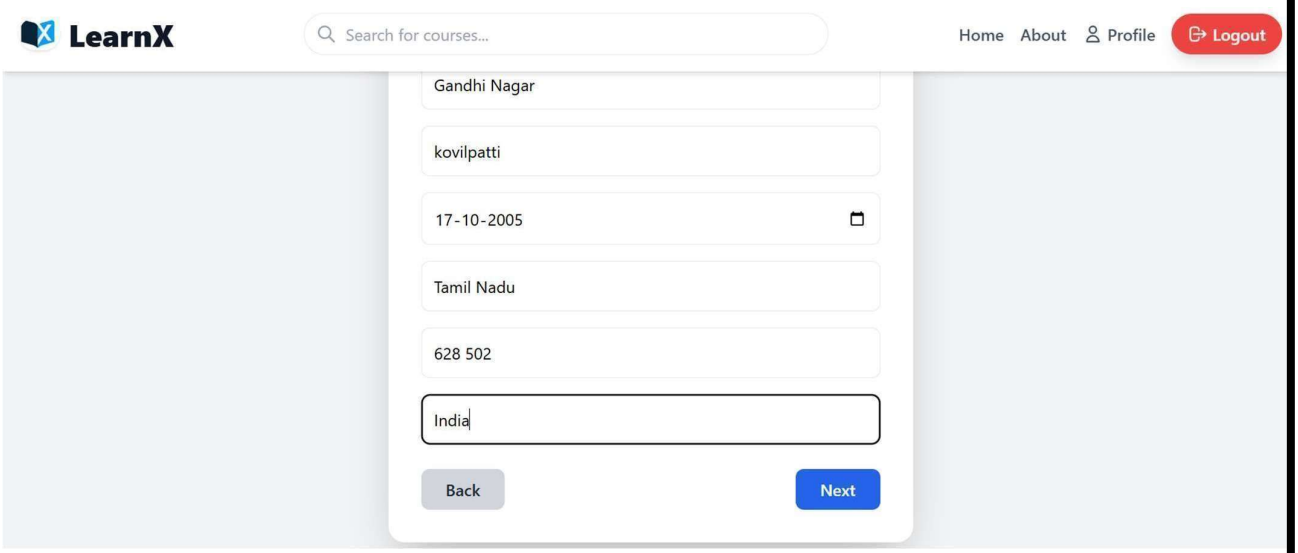


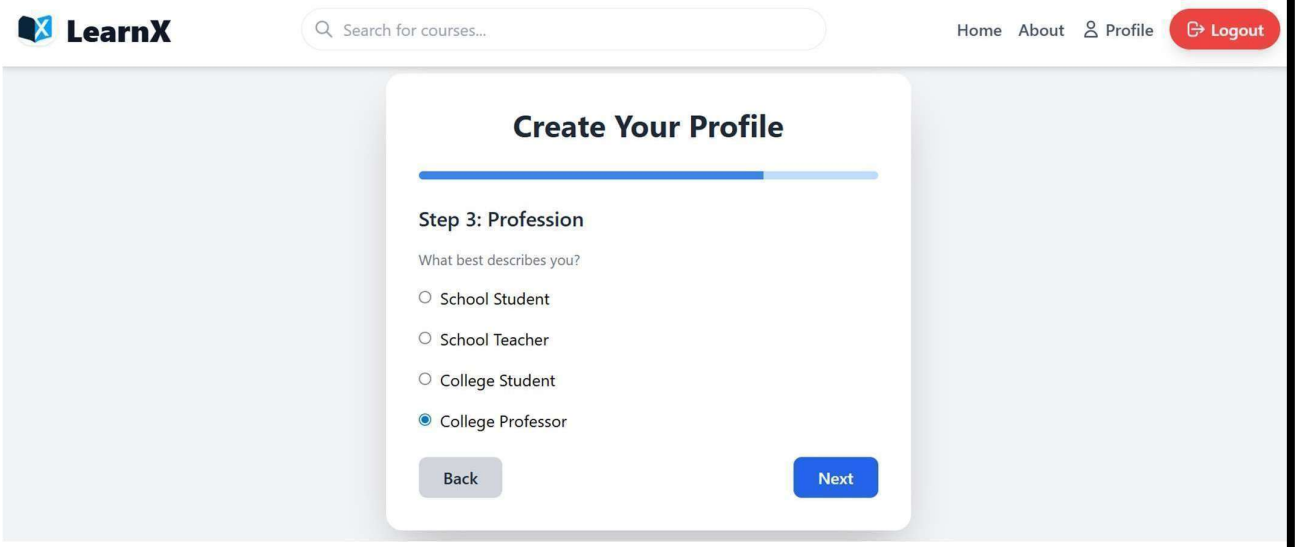


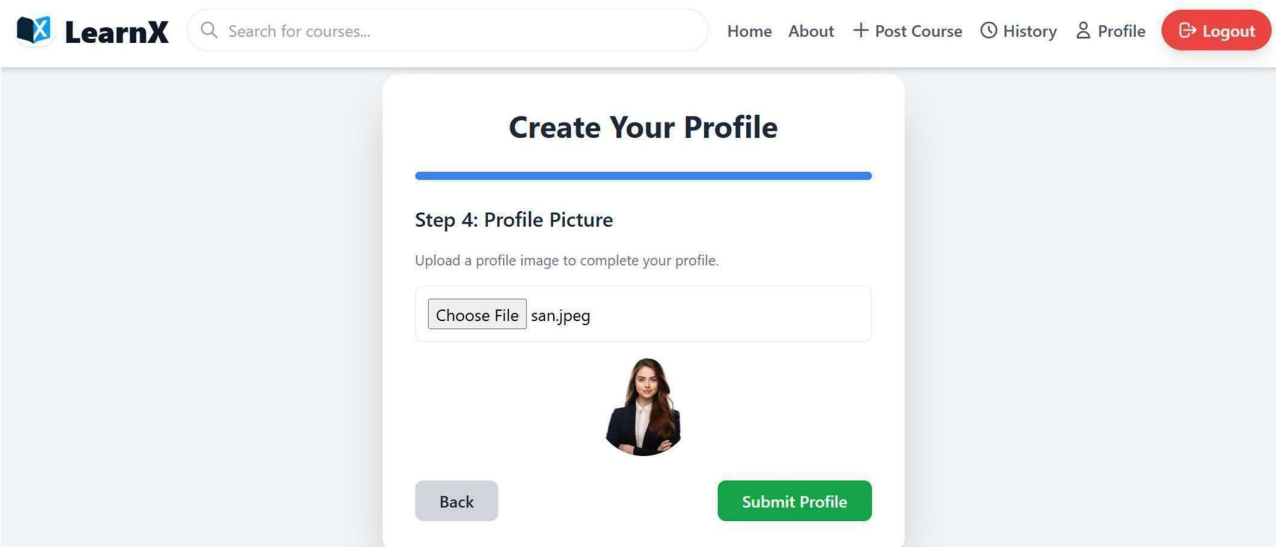
PROFILE CREATION PAGE

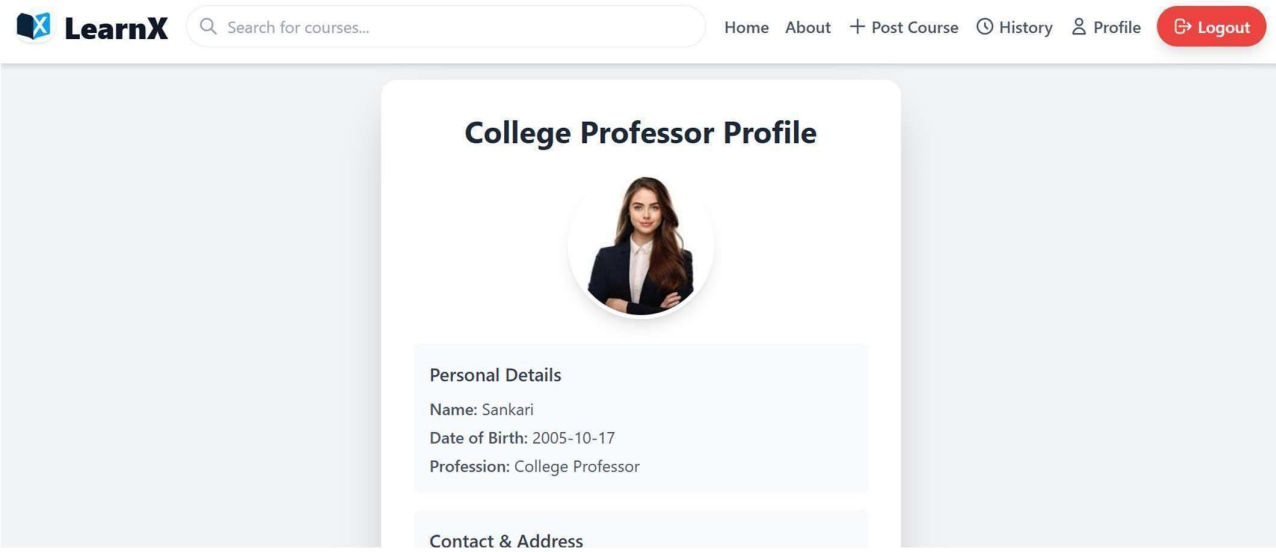


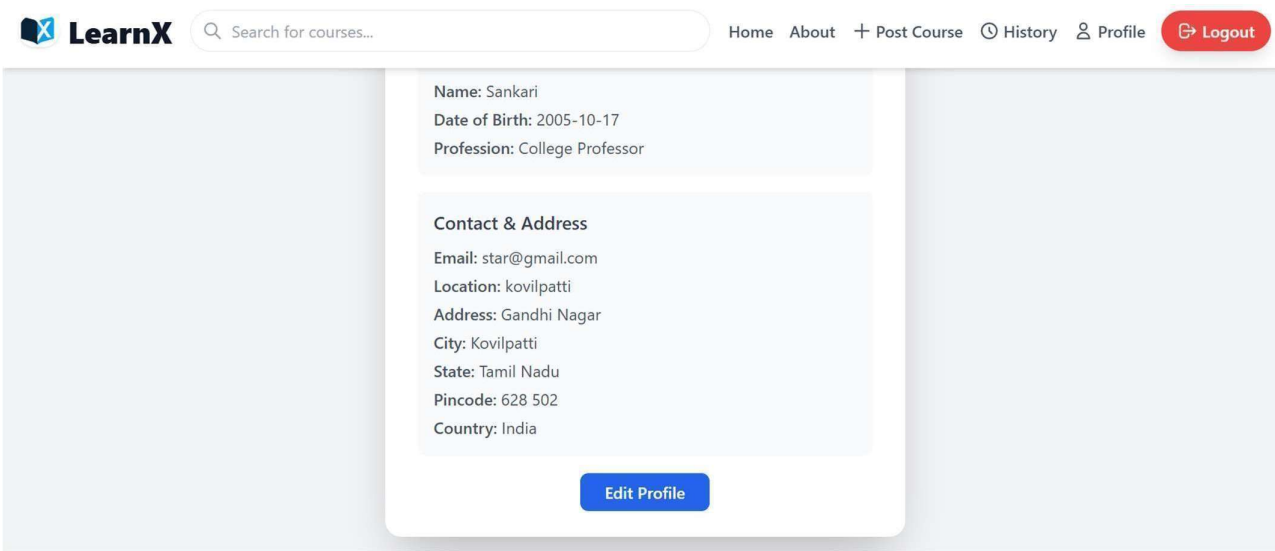



LearnX
Search for courses...
Home
About
Profile
Logout
Gandhi Nagar
kovilpatti
17-10-2005
Tamil Nadu
628 502
India
Back
Next

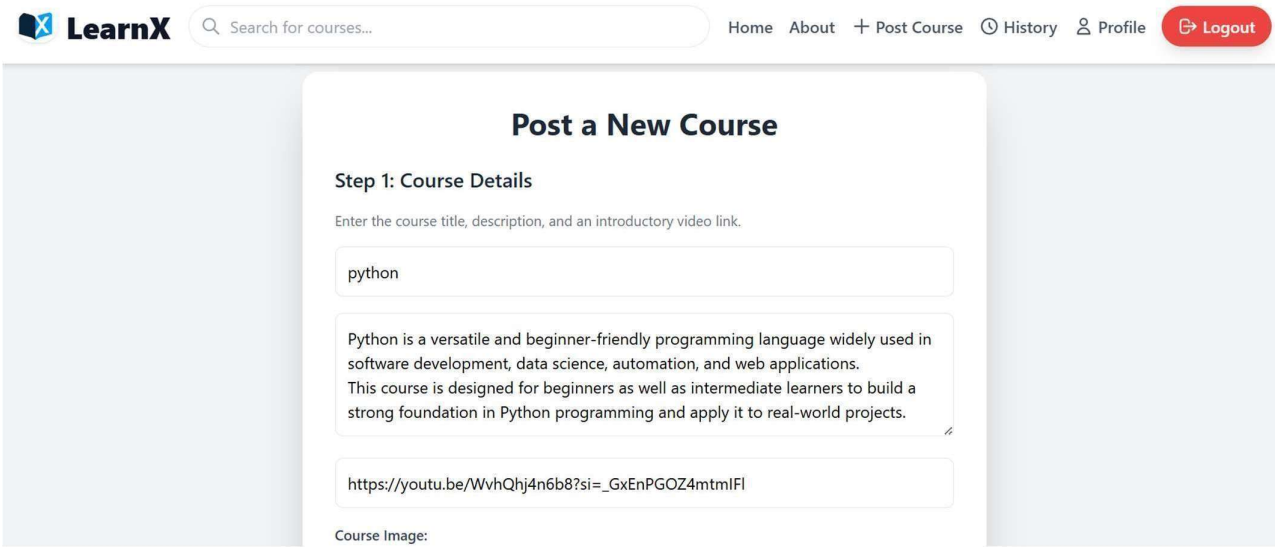

LearnX
Search for courses...
Home
About
Profile
Logout
Create Your Profile
Step 3: Profession
What best describes you?
School Student
School Teacher
College Student
College Professor
Back
Next

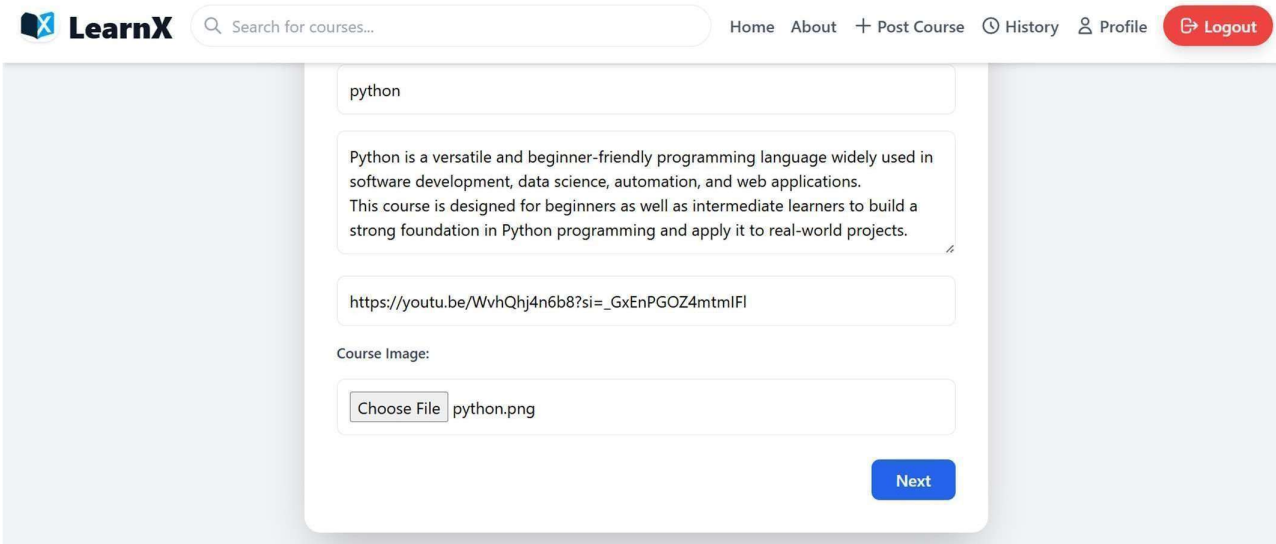

LearnX
Search for courses...
Home
About
Post Course
History
Profile
Logout
Create Your Profile
Step 4: Profile Picture
Upload a profile image to complete your profile.
Choose File
san.jpeg
Back
Submit Profile


LearnX
Search for courses...
Home
About
Post Course
History
Profile
Logout
College Professor Profile
Personal Details
Name: Sankari
Date of Birth: 2005-10-17
Profession: College Professor
Contact & Address



POST COURSE PAGE





LIST OF COURSES PAGES

C Programming Language

Professor: **sound** | kovilpatti

This course is designed to help students learn and master the C programming language from scratch. It covers everything from the...

View More Details

C++

Professor: **banu** | kovilpatti

This course provides a comprehensive introduction to C++ programming, covering everything from fundamental concepts to...

View More Details

Artificial Intelligence (AI)

Professor: **vidhya** | kovilpatti

This Artificial Intelligence (AI) course is designed to provide students with a strong foundation in AI concepts, algorithms, and...

View More Details

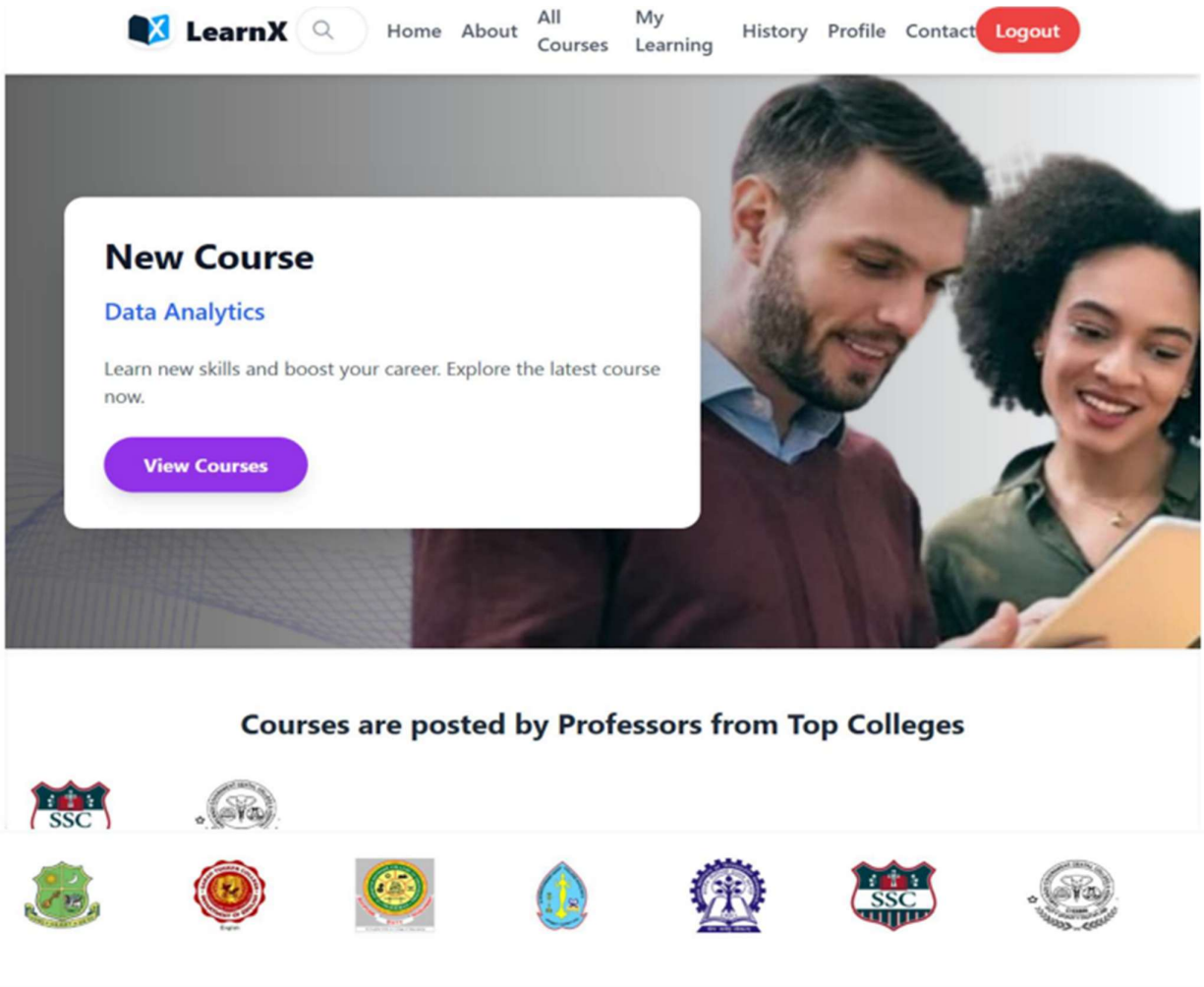
RDBMS

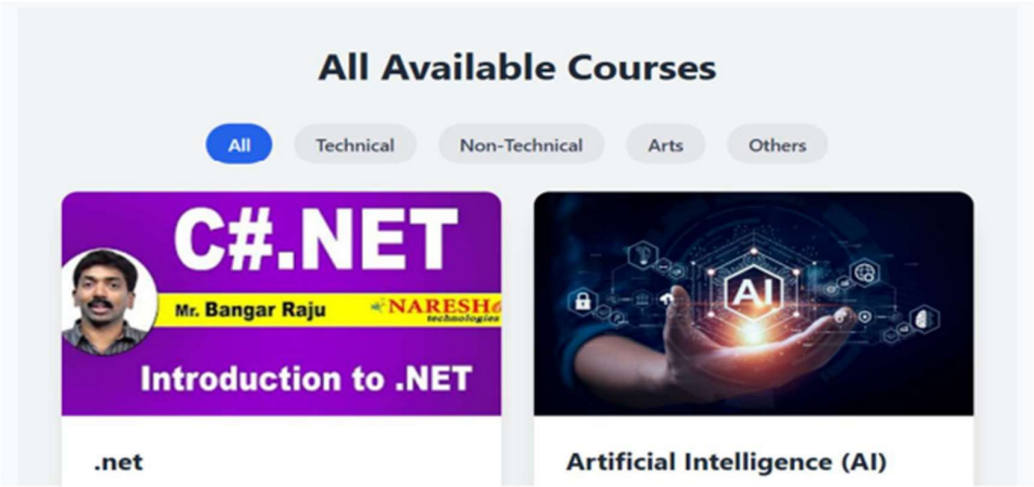
Professor: **vidhya** | kovilpatti

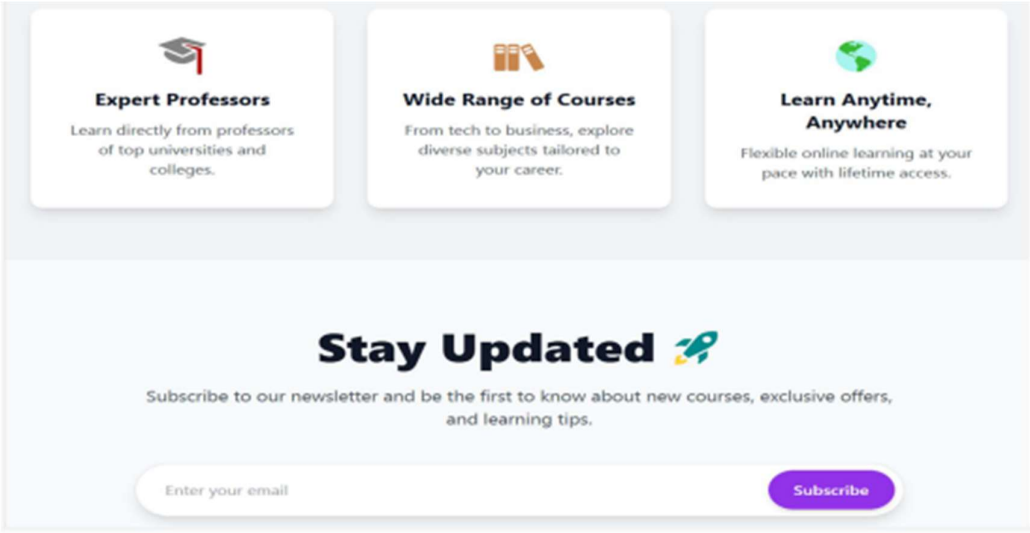
This course provides a comprehensive understanding of Relational Database Management Systems (RDBMS), focusing on...

View More Details

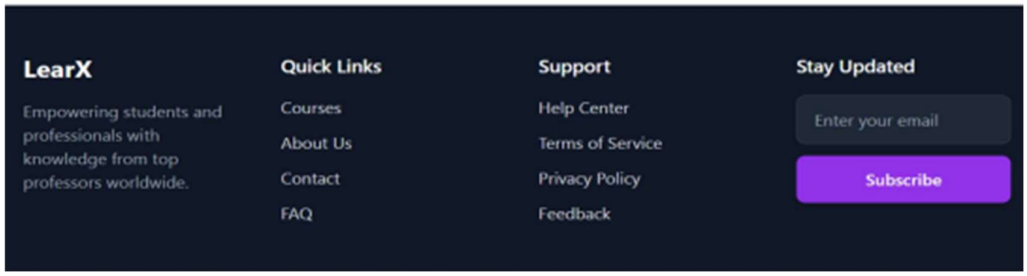
STUDENT LEARNING PAGE



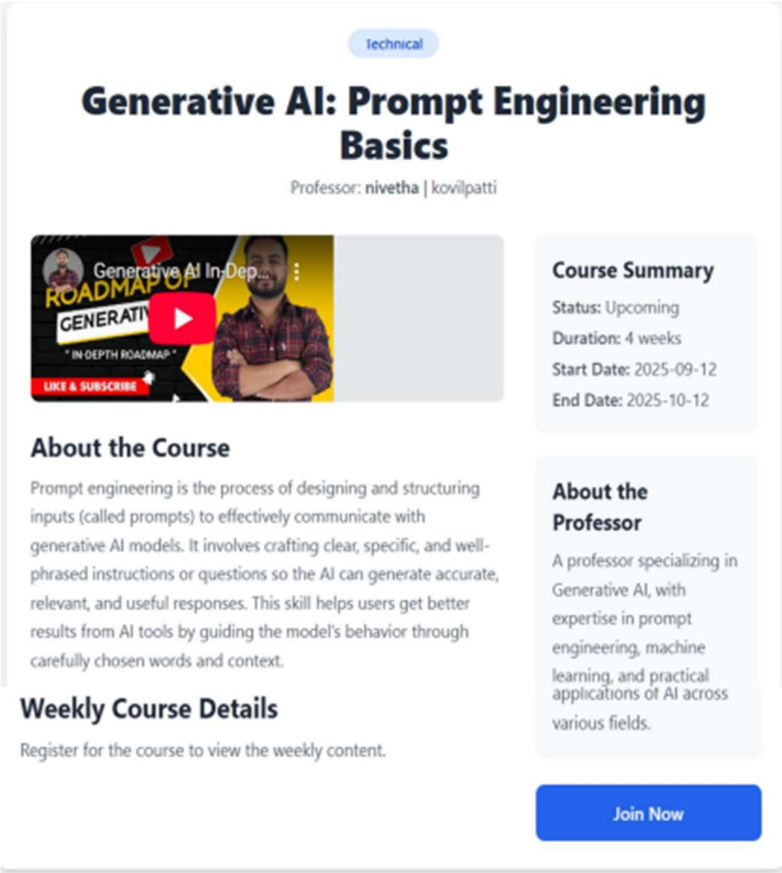



Expert Professors
Learn directly from professors of top universities and colleges.
Wide Range of Courses
From tech to business, explore diverse subjects tailored to your career.
Learn Anytime, Anywhere
Flexible online learning at your pace with lifetime access.
Stay Updated
Subscribe to our newsletter and be the first to know about new courses, exclusive offers, and learning tips.
Enter your email
Subscribe

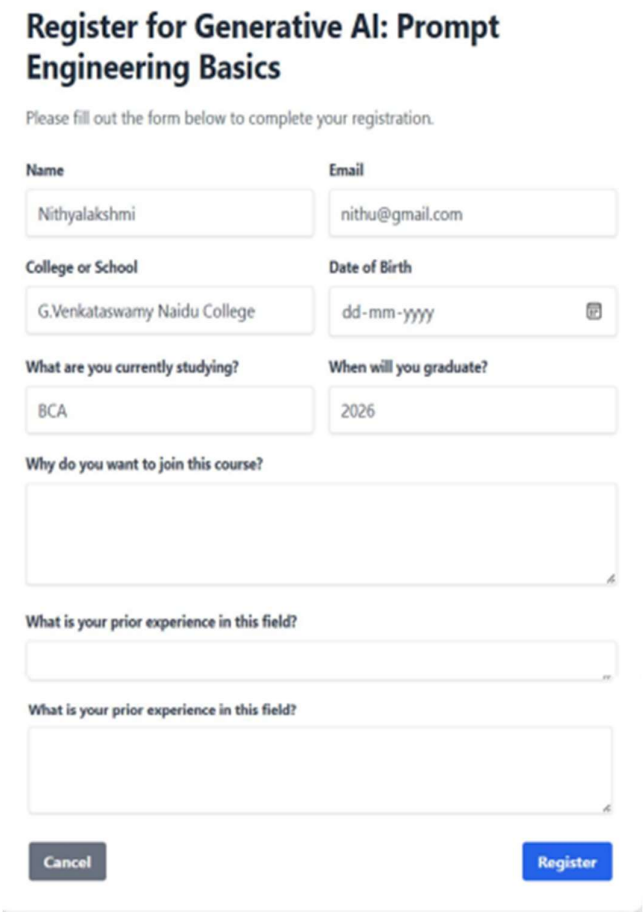
What Students Say
"LearX helped me crack my dream job interview. The professors explain concepts so clearly!"
— Ananya, CS Student
"The best part is learning directly from top college professors. Feels like a real classroom."
— Rahul, MBA Student
"Flexible courses and lifetime access helped me balance learning with my job."
— Priya, Working Professional


LearX
Empowering students and professionals with knowledge from top professors worldwide.
Quick Links
Courses
About Us
Contact
FAQ
Support
Help Center
Terms of Service
Privacy Policy
Feedback
Stay Updated
Enter your email
Subscribe

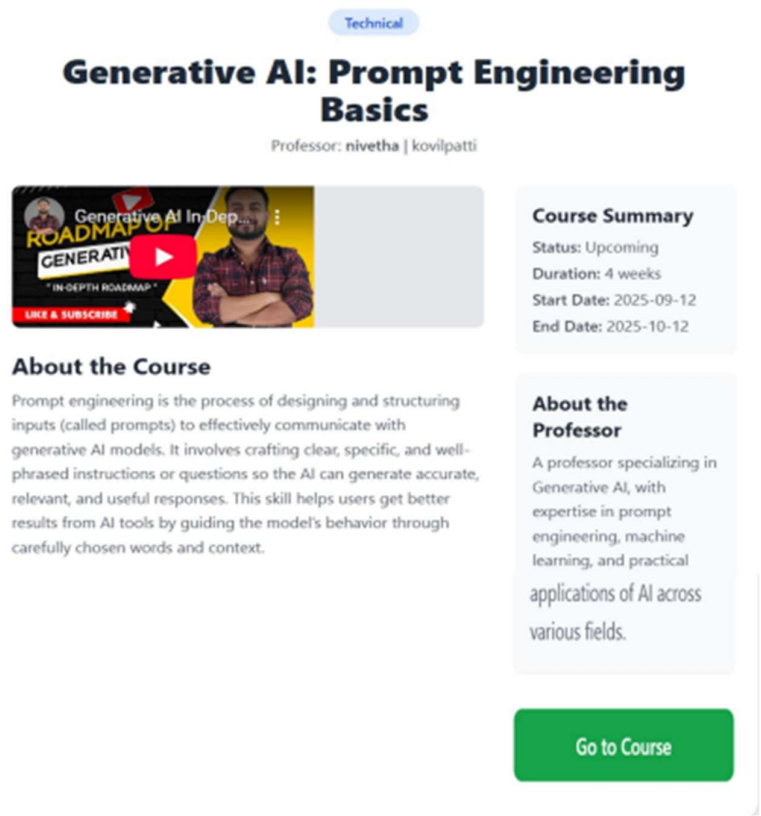
COURSE DETAILS PAGE



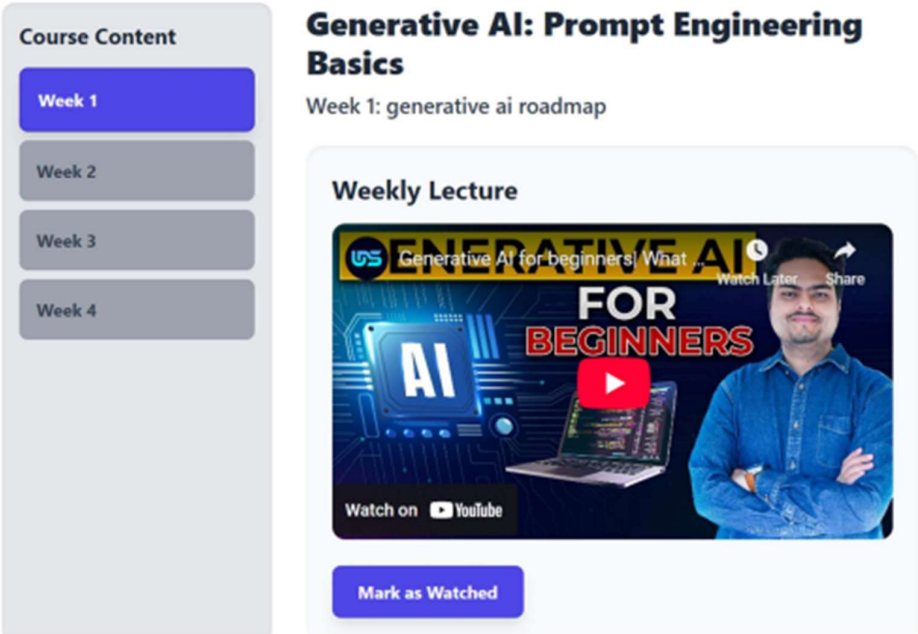
COURSE REGISTERATION PAGE



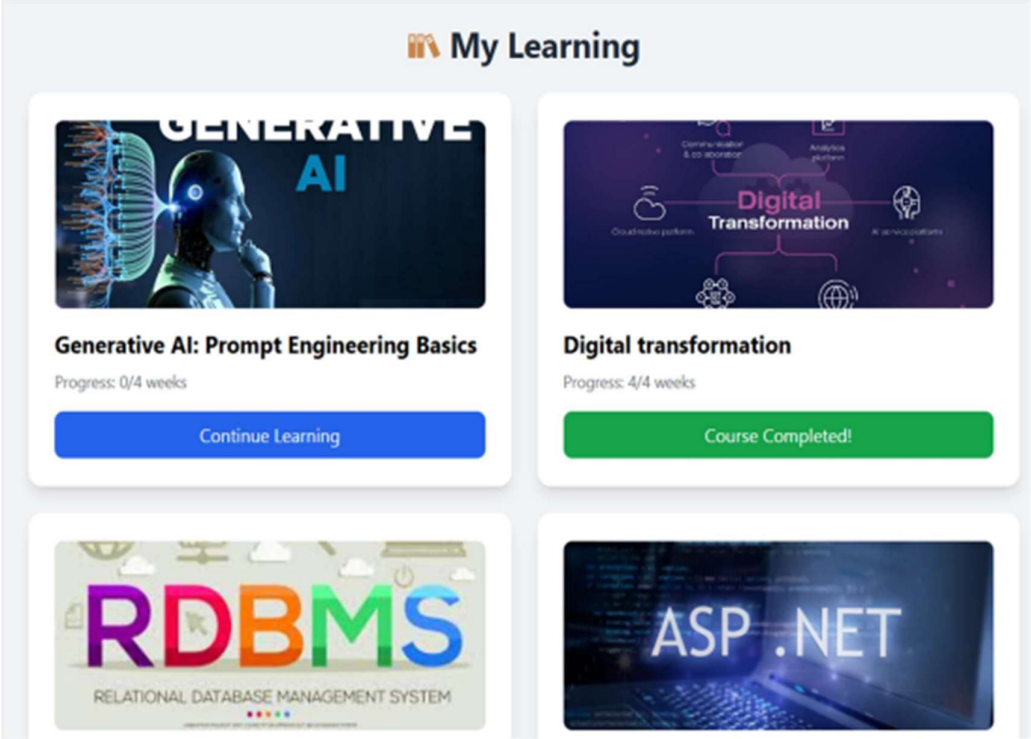
COURSE LEARNING PAGE



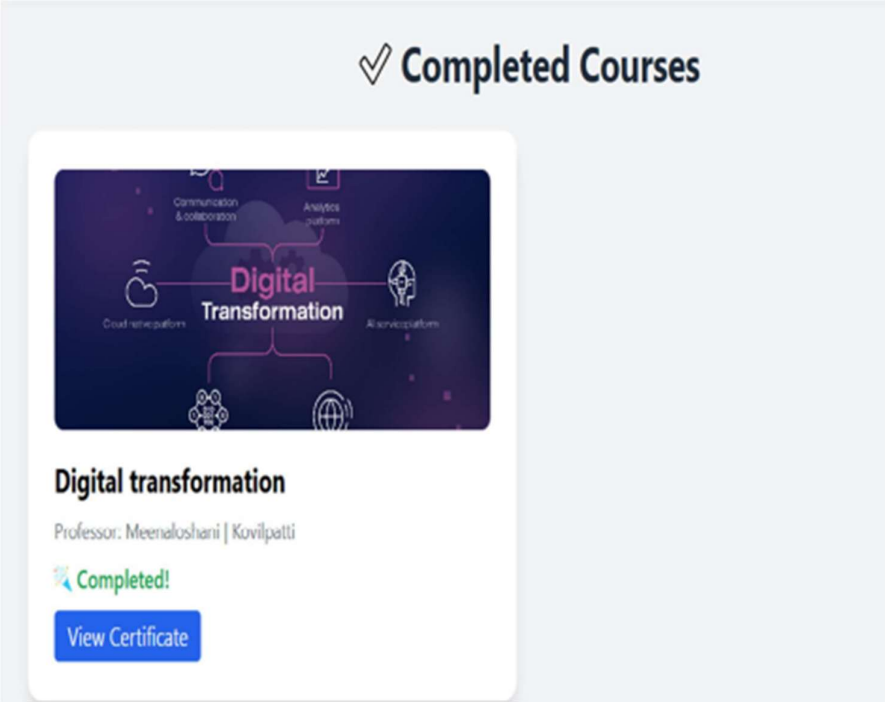
WEEK ASSIGNMENT PAGE



LEARNING HISTORY



COURSE COMPLETED



10. CONCLUSION

The LearnX platform is designed as a comprehensive digital learning companion that streamlines education for students, teachers, and administrators. With features like course management, live class scheduling, performance tracking, and OTP-based authentication, it ensures secure and efficient interactions across the platform. Its role-based navigation provides tailored access, ensuring that each user experiences a personalized and seamless interface.

Beyond classroom learning, LearnX empowers students with access to skill-development resources, practice tests, and progress analytics, enabling them to prepare effectively for both academics and real-world challenges. Teachers benefit from simplified tools for assignment management, grading, and attendance tracking, while administrators can oversee overall academic operations from a centralized dashboard.

By integrating academic support with advanced digital features, LearnX goes beyond being just a learning app—it becomes a career-ready platform. Its scalable and secure structure enhances collaboration, minimizes manual tasks, and builds a bridge between students' academic journeys and their future aspirations, driving digital transformation in modern education.

11 .FUTURE ENHANCEMENT

In the future, LearnX can be expanded with several advanced features to provide a richer learning:

- **AI-Powered Personalized Learning** – Use machine learning to recommend courses and study materials based on student progress and interests.
- **Gamification Features** – Add badges, leaderboards, and rewards to motivate students.
- **Live Classes & Webinars** – Enable teachers to host real-time classes and Q&A sessions.
- **Mobile App Integration** – Develop Android and iOS applications for better accessibility.
- **Advanced Analytics** – Provide teachers with insights into student performance and progress through detailed dashboards.
- **Multi-Language Support** – Allow students to learn in their preferred language for wider accessibility.
- **Integration with Job Platforms** – Connect course certifications with job portals to help students with career opportunities.

12 REFERENCE

- **React Documentation** – https://react.dev/
- **Firestore Database** – https://firebase.google.com/docs/firestore
- **Flask Web Framework** – https://flask.palletsprojects.com/
- **Axios (API Calls in React)** – https://axios-http.com/docs/intro
- **Material-UI (React UI Components)** – https://mui.com/
- **Tailwind CSS** – https://tailwindcss.com/docs
- **Google Firebase Authentication** – https://firebase.google.com/docs/auth
- **JavaScript (MDN Docs)** – https://developer.mozilla.org/en-
- **React Router** – https://reactrouter.com/en/main